This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

Speeding-up Graph Algorithms via Parallel Randomized Clique Partitioning

Akshar Chavan, *Student Member, IEEE,* Sanaz Rabinia, *Student Member, IEEE,* Daniel Grosu, *Senior Member, IEEE,* Marco Brocanelli, *Senior Member, IEEE*

Abstract—Large-scale graph analytics face significant performance challenges due to the size and structural complexity of modern graphs, particularly for algorithms that rely on global connectivity. To address these challenges, a variety of graph reduction and restructuring techniques have been proposed, including sparsification, coarsening, and lossless compression. While sparsification and coarsening improve efficiency by reducing graph size, they often compromise structural fidelity or require reconstruction. In contrast, lossless approaches preserve exact connectivity but are typically sequential or introduce overheads that limit their scalability and direct applicability to downstream algorithms. In this paper, we present Parallel Randomized Clique Partitioning (Par-RCP), a randomized parallel algorithm for lossless graph restructuring based on bipartite clique partitioning. Our approach transforms the graph into a reduced representation that preserves complete path information, enabling downstream algorithms to operate directly on the restructured graph without additional reconstruction overhead. We first develop a randomized sequential algorithm, Seq-RCP, with a running time of $O(mn^{\delta})$, where m and n denote the number of edges and vertices of the input graph and $0 \leq \delta \leq 1$ is a constant. We then design Par-RCP, which achieves a theoretical parallel running time of $O(n^{\delta} \log^2 n)$ on an EREW PRAM with n^2 processors. Experimental results on large-scale graphs with billions of edges show that Par-RCP reduces the number of edges by up to $6.69\times$ and achieves speedups of up to 29.25 over the sequential baseline. Using the restructured graph further accelerates downstream algorithms, yielding up to 2.06 speedup for Dinitz's algorithm on large-scale graphs and up to 213.34 for all-pairs shortest paths on real-world sparse datasets from SuiteSparse. These results demonstrate the scalability and practical effectiveness of our approach for large-scale graph analytics.

1 Introduction

Graphs are fundamental tools in mathematics and computer science, used to model pairwise relationships across diverse domains such as communication networks, transportation systems, and social networks. However, many real-world applications involve extremely large graphs, leading to high processing costs and substantial storage requirements. Addressing these challenges requires methods that exploit the structural properties of graphs to accelerate computationally intensive tasks and scale to increasing data sizes. To mitigate the overhead of processing dense graphs, restructuring techniques have emerged as an alternative to traditional sparsification. In particular, Feder and Motwani [1] pioneered lossless graph restructuring using bipartite clique partitioning. Their work demonstrated that global connectivity-dependent algorithms, such as matching and all-pairs shortest paths (APSP), can be accelerated by preserving all path information in a more compact representation. However, Feder and Motwani's work is largely theoretical, and its practical performance on very large or dense graphs has not been thoroughly studied. More recently, Chavan et al. [2] proposed a deterministic clique partitioning-based algorithm for graph restructuring inspired by the Feder-Motwani framework. While that approach demonstrates the feasibility of deterministic restructuring, it remains inherently sequential and struggles to scale to massive graphs. Furthermore, other existing approaches [3], [4], [5], [6] also employ graph transformations, but they are primarily designed for query-based settings where partial connectivity is sufficient. For algorithms that require complete graph connectivity information, these methods typically rely on costly reconstruction steps.

In this paper, we bridge these gaps by introducing Parallel Randomized Clique Partitioning (Par-RCP), a lossless graph restructuring algorithm designed to accelerate global connectivity-dependent tasks. Our method efficiently reduces the number of edges while preserving complete path information, where path information refers to the existence of paths between vertices, allowing direct use of the restructured graph in subsequent algorithms such as matching and all-pairs shortest paths. This design eliminates the need for costly reconstruction, thereby making the acceleration of these computationally intensive tasks significantly more practical and scalable. Crucially, Par-RCP achieves substantially higher reduction in the number of edges and faster restructuring times than prior lossless methods, meeting the performance demands of massive real-world datasets.

To achieve this, we first design a randomized sequential clique partitioning-based algorithm (Seq-RCP) for graph restructuring. This forms the foundational base for the design of our parallel algorithm. Both the sequential and parallel versions strategically select cliques to determine the graph's partitioning, yielding approximately the same reduction in the number of edges of the original graph. We demonstrate that this reduction is comparable to, or even surpasses, that of Feder-Motwani's approach [1] and recent

- *A. Chavan and M. Brocanelli are with the Dept. of Electrical and Computer Engineering, Ohio State Univ., Columbus, OH 43210. E-mail: chavan.43@buckeyemail.osu.edu, brocanelli.1@osu.edu. S. Rabinia and D. Grosu are with the Dept. of Computer Science, Wayne State Univ., Detroit, MI 48202. E-mail: {srabin, dgrosu}@wayne.edu.*

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

practical deterministic approaches [2]. A key contribution is the parallel algorithm's significant speedup relative to its sequential counterpart for the restructuring process itself, enabling efficient pre-processing of massive graphs for downstream global connectivity analysis.

These limitations of existing graph restructuring techniques motivate the design of our randomized clique-partitioning algorithms for graph restructuring. We develop both a sequential algorithm, Seq-RCP, and a parallel algorithm, Par-RCP. Our key contributions are:

1) We introduce a randomized clique-partitioning framework for lossless graph restructuring that enables more effective discovery of cliques compared to prior approaches, leading to improved edge reduction.
2) We show that our algorithms achieve an average reduction in the number of edges of $O(\log n)$, while empirically attaining significantly higher compression, with up to $3.63\times$ greater edge reduction compared to prior state-of-the-art methods.
3) We demonstrate that Par-RCP is highly scalable in practice, achieving up to 89.35 speedup over existing approaches on large graphs with up to ≈ 966M edges.
4) We provide theoretical guarantees for our algorithms: Seq-RCP runs in $O(mn^{\delta})$ time, while Par-RCP runs in $O(n^{\delta}\log^2 n)$ time with high probability on an EREW PRAM with n^2 processors, where $0 \leq \delta \leq 1$ is a constant.
5) We show that Par-RCP achieves strong parallel performance in practice, with up to 29.25 speedup over Seq-RCP, while maintaining high edge reduction.
6) We demonstrate the effectiveness of our restructuring approach in accelerating downstream applications: the Dinitz algorithm for maximum cardinality matching achieves up to 2.06 speedup, and All-Pairs Shortest Paths achieves up to 213.34 speedup when applied to the restructured graph.

2 Related Works

Existing research to optimize large-scale graph processing generally falls into two categories: graph sparsification and graph restructuring. While sparsification reduces graph density by removing edges (often sacrificing exactness), restructuring techniques transform the graph into a more efficient representation while preserving critical connectivity information.

Lossless Graph Restructuring for Speeding-up Graph Algorithms. Feder and Motwani [1] pioneered a deterministic, theory-driven approach to lossless graph restructuring based on bipartite clique partitioning, with the goal of accelerating global connectivity-dependent graph algorithms such as bipartite matching [7], edge connectivity [8], and vertex connectivity [9]. Their algorithm achieves a running time of $O(mn^{\delta}\log^2 n)$ and provides strong theoretical guarantees, while also establishing the NP-completeness of finding optimal clique partitions. Building on this foundation, Chavan et al. [2] proposed a deterministic, sequential clique partitioning-based algorithm focused on practical performance. Their approach improves clique extraction efficiency and achieves stronger edge reduction in practice, making lossless restructuring more applicable to real-world graphs. However, both approaches rely on sequential clique extraction processes and structured vertex selection strategies, which introduce inherent dependencies that limit parallelization. As a result, their scalability remains constrained when applied to massive graphs.

Alternative Graph Transformation Techniques. Other graph transformation techniques pursue different objectives. Lossless compression methods [10] primarily focus on reducing storage space [11] or accelerating specific queries such as triangle counting or community detection [3], [4], [5], [6]. While these approaches can achieve high compression ratios, the resulting representations are often not directly suitable for global graph algorithms and may require costly reconstruction.

Compressed Graph Processing. Several graph processing systems execute algorithms directly on compressed graph representations. Ligra+ [12] extends the Ligra shared-memory framework with compressed adjacency encodings, reducing graph storage while preserving its programming interface. CompressGraph [13] uses rule-based compression to represent repeated graph structure and enables computation reuse during graph processing. Par-RCP differs from these approaches by performing a structural graph transformation: edge-disjoint bicliques are replaced with auxiliary vertices, reducing the number of explicit edges processed by downstream algorithms. Thus, while Ligra+ focuses on compressed adjacency storage and CompressGraph on rule-based compression and computation reuse, Par-RCP focuses on biclique-based structural edge reduction. A direct performance comparison would require evaluating the systems under a common execution framework, algorithm set, and hardware configuration.

Hardware-Aware Graph Processing. HashGrid [14] combines graph partitioning with an FPGA-based architecture to reduce irregular memory-access overheads during graph processing. In contrast, Par-RCP reduces the explicit edge set through biclique-based restructuring before downstream execution. These approaches are complementary, and combining structural graph reduction with hardware-aware processing is an interesting direction for future work.

Randomized and Parallel Graph Processing. Randomized and parallel techniques have been explored extensively in broader graph processing contexts. Prior work has leveraged randomization for graph summarization [15], [16] and parallelism for improving performance in shared-memory and large-scale systems. Recently, significant efforts have focused on parallelizing specific downstream tasks or handling graphs in distributed computing environments. This includes advancements in communication-efficient, hardware-accelerated graph processing [17] and distributed adaptive partitioning for massive networks [18]. Furthermore, extensive research has addressed accelerating shortest-path queries on large-scale networks using parallel algorithm templates and multi-objective updates [19], [20].

While these state-of-the-art approaches offer massive performance gains for specific operational models or dynamic streaming contexts [21], they typically operate directly on the original graph topology. Our lossless graph restructuring approach is orthogonal and complementary to these methods. We introduce a randomized parallel restruc-

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

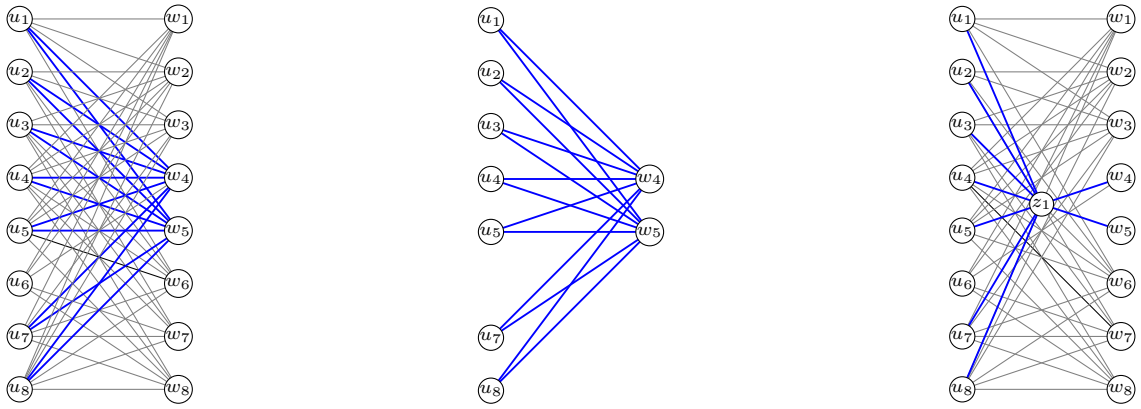


Fig. 1: (a) Bipartite clique C_q (with blue edges) in a given bipartite graph $G(U, W, E)$; (b) Extracted bipartite clique C_q from G; and (c) tripartite graph $G^*(U, W, Z, E^*)$ that replaces the bipartite clique in the restructured graph.

turing algorithm that enables concurrent clique extraction. By relaxing rigid selection constraints and reducing sequential dependencies, our approach achieves strong edge reduction while significantly improving scalability, providing a practical lossless graph representation useful for large-scale graph analytics.

3 Background

Graph restructuring aims to transform a graph into a more compact representation while preserving its essential connectivity properties, enabling faster execution of downstream graph algorithms. A central approach to lossless graph restructuring is based on identifying structured subgraphs, such as bipartite cliques, and replacing them with more compact representations. In this work, we build on the clique partitioning-based restructuring framework introduced by Feder and Motwani [1] (referred to as FM) and its subsequent practical realization by Chavan et al. [2] (referred to as CPGR). Both approaches operate on a bipartite graph $G = (U, W, E)$ and iteratively extract δ-*cliques*, i.e., complete bipartite subgraphs with left and right partitions $U_q \subseteq U$ and $W_q \subseteq W$ of size $\lceil n^{1-\delta} \rceil$ and $\hat{k} = \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$, respectively, for some constant $0 \leq \delta \leq 1$. Choosing $\hat{k} = \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$ guarantees the existence of such δ-cliques in graphs with n vertices and $\hat{m}$ edges, and is used in both FM and CPGR. Each extracted clique C_q is replaced by a newly introduced vertex z_q that connects to all vertices in U_q and W_q, transforming G into a tripartite graph $G^* = (U, W, Z, E^*)$, where Z is the set of added vertices.

Figure 1 illustrates this restructuring process. A δ-clique C_q in the original graph is replaced by a new vertex z_q, reducing the number of edges from $|U_q| \cdot |W_q|$ to $|U_q| + |W_q|$. By iteratively applying this transformation, the total number of edges in the graph is reduced while preserving its structural properties. A key property of this framework is that it is lossless, i.e., for any pair of vertices, path connectivity in the original graph is preserved in the restructured graph G^*. This allows downstream algorithms, such as matching and all-pairs shortest paths, to operate directly on G^* without requiring reconstruction. The FM algorithm provides strong theoretical guarantees, including an $O(\log n)$ reduction in the number of edges with a running time of $O(mn^\delta \log^2 n)$. Building on this foundation, CPGR improves practical performance by enabling more efficient clique extraction and allowing multiple δ-cliques to be identified within an iteration, leading to stronger edge reduction in real-world graphs.

However, both FM and CPGR exhibit limitations when applied to large-scale graphs. In particular, FM relies on iterative extraction of δ-cliques, where each step depends on the current state of the graph. While CPGR improves upon this by enabling the extraction of multiple δ-cliques within an iteration, the overall restructuring process remains inherently sequential due to dependencies on graph updates across iterations. As a result, these approaches cannot extract all possible δ-cliques concurrently and are unable to fully exploit parallel computation. This sequential dependency becomes a bottleneck for massive graphs, motivating the need for a parallel and more scalable restructuring approach.

4 Proposed Algorithms

In this section, we present the proposed sequential and parallel randomized clique partitioning-based graph restructuring algorithms.

4.1 Sequential Randomized Clique Partitioning-Based Algorithm for Graph Restructuring (Seq-RCP)

In this section, we present the design of our Sequential Randomized Clique Partitioning-based (Seq-RCP) algorithm for graph restructuring. For a given input bipartite graph $G(U, W, E)$, Seq-RCP iteratively finds bipartite cliques in G, and restructures it until finding new cliques no longer aids in the reduction of the number of edges in the restructured graph G^*. The size of the right partition of cliques is determined by $\hat{k}$, where Seq-RCP randomly selects $\hat{k}$ vertices $w_j \in W$, iteratively. The selection of vertices in the left partition is determined by finding all the common neighbours for those $\hat{k}$ randomly selected vertices in W. The partition of G into bipartite cliques

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

Algorithm 1 Seq-RCP: Sequential Randomized Clique Partitioning-Based Algorithm for Graph Restructuring

Input: A: adjacency matrix of $G = (U, W, E)$
n: number of vertices of G, $n = |W| = |U|$
m: number of edges of G.
δ: constant, such that, $0 \leq \delta \leq 1$.

1: $\mathcal{C} \leftarrow \varnothing$
2: $q \leftarrow 1$
3: $\hat{m} \leftarrow m$
4: $\hat{k} \leftarrow \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$
5: **while** $\hat{k} > 1$ **do**
6: $\quad \widehat{W} \leftarrow W$
7: $\quad$ **while** $|\widehat{W}| \neq 0$ **do**
8: $\quad\quad W_q \leftarrow \varnothing$
9: $\quad\quad$ **for** $i = 1, \ldots, \hat{k}$ **do**
10: $\quad\quad\quad$ Randomly select w from $\widehat{W}$
11: $\quad\quad\quad \widehat{W} = \widehat{W} \setminus \{w\}$
12: $\quad\quad\quad W_q \leftarrow W_q \bigcup \{w\}$
13: $\quad\quad U_q \leftarrow \{u_i \in U \mid W_q \subseteq N(u_i)\}$
14: $\quad\quad C_q \leftarrow \{(U_q, W_q)\}$
15: $\quad\quad$ Update A, by removing edges $(U_q \times W_q) \in C_q$
16: $\quad\quad \hat{m} \leftarrow \hat{m} - |(U_q \times W_q)|$
17: $\quad\quad \mathcal{C} \leftarrow \mathcal{C} \bigcup \{C_q\}$
18: $\quad\quad q \leftarrow q + 1$
19: $\quad \hat{k} \leftarrow \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$
20: For each clique in $\mathcal{C} = \{C_1, \ldots, C_q\}$ add edges from all the vertices in the left partition U_q to an additional vertex z_q, and from z_q to all the vertices in the right partition W_q. This would restructure the graph by replacing $|U_q| \times |W_q|$ edges with $|U_q| + |W_q|$ edges.
21: The remaining edges in G are trivial cliques and are added in the restructured graph G^*.
22: **return** Restructured graph G^*

guarantees that each edge in G is in exactly one clique, i.e., $E(C_i) \cap E(C_j) = \emptyset, \forall i, j$ and $i \neq j$. Seq-RCP is given in Algorithm 1. The input of Seq-RCP consists of the adjacency matrix A, number of vertices n, number of edges m of G, and a constant δ, $0 \leq \delta \leq 1$.

Initialization. For the given bipartite graph $G(U, W, E)$, Seq-RCP initializes the set of bipartite cliques $\mathcal{C}$ to empty set, the index q of the bipartite cliques extracted from G to 1, and the number $\hat{m}$ of remaining edges in G to m (Lines 1-3). Then in Line 4, it computes the size of the right partition of the bipartite clique, $\hat{k}$, which guarantees the existence of a δ-clique with $\hat{k}(n, \hat{m}, \delta)$.

The algorithm's core functionality is organized into three main phases: vertex selection, clique extraction, and graph restructuring. For efficient graph restructuring, our objective is to ensure that the number of edges in the resulting restructured graph G^* is significantly lower than that of the given input graph G. This is achieved by iteratively replacing complete bipartite subgraphs (δ-cliques) with a tripartite subgraph that has fewer edges than the original bipartite subgraph it replaces. To achieve this, the algorithm proceeds iteratively (as shown in Lines 5-19 of Algorithm 1) as long as conditions for effective restructuring are met. Specifically, the iteration continues while the size of the right partition for potential δ-cliques, $\hat{k}$, remains greater than 1. This condition ensures that extracted cliques contribute to a net reduction or, at minimum, maintenance of the total edge count in the restructured graph G^*.

Vertex Selection. The initial step involves selecting vertices for the right partition of a δ-clique. The exact number of vertices to be selected is determined by the parameter $\hat{k}$. A key objective is to ensure that the right partitions of distinct δ-cliques are disjoint (i.e., $W_i \cap W_j = \emptyset$ for $i \neq j$, where W_i and W_j are the right partitions of δ-cliques indexed i and j respectively, and $i, j \in [1, |\mathcal{C}|]$). To achieve this, our algorithm randomly selects $\hat{k}$ vertices from the set of currently available vertices $\widehat{W}$. These selected vertices are then designated as candidates for the right partition W_q of the current δ-clique, indexed by q. To enforce disjointness, these selected vertices are immediately removed from $\widehat{W}$ for any further selection within the remainder of the current iteration (as detailed in Lines 7-12 of Algorithm 1).

Clique Extraction. Once the right partition W_q for a potential δ-clique C_q has been determined, the next step is to form the complete δ-clique. This involves identifying the common neighbors of all vertices in W_q to constitute the left partition U_q. A crucial property of our restructuring approach, ensuring a truly distinct partition of the graph's edges, is that no two extracted δ-cliques share any edges (i.e., $E(C_i) \cap E(C_j) = \emptyset$ for $i \neq j$, where $E(C_i)$ and $E(C_j)$ represent the edge sets of distinct δ-cliques C_i and C_j). To maintain this edge disjointness and accurately reflect the remaining graph structure, the edges forming the newly extracted δ-clique C_q are immediately removed from the graph. This edge removal directly updates $\hat{m}$, the count of remaining edges in G that have not yet been assigned to any δ-clique. The updated value of $\hat{m}$ is then used to recalculate $\hat{k}$ for the subsequent iteration, ensuring that the parameters for future δ-clique extractions are based on the current available edges.

Specifically, in Lines 13-14, Algorithm 1 identifies the common neighbors of the selected right partition W_q to form the bipartite clique C_q. Subsequently, in Lines 15-18, it updates $\hat{m}$ (the number of remaining edges in G), adds the newly formed bipartite clique C_q to the set of all extracted bipartite cliques $\mathcal{C}$, and increments the index q. Finally, in Line 19, after processing the current clique, Seq-RCP updates $\hat{k}$ using the recalculated value of $\hat{m}$ before continuing to the next iteration of the main loop (Line 5).

Graph Restructuring. Once all δ-cliques have been extracted, the final step involves constructing the restructured graph G^*. The primary objective of this process is to ensure that G^* has a significantly lower number of edges than the original input graph G, while preserving path information. To do so, for each extracted clique C_q in the set $\mathcal{C}$ (obtained from the partitioning), the algorithm performs a specific transformation. In Line 20, it adds edges connecting all vertices in the left partition U_q to a newly introduced additional vertex z_q, and subsequently adds edges from this new vertex z_q to all vertices in the right partition W_q. This transformation effectively replaces the $|U_q| \times |W_q|$ edges of the original complete bipartite clique C_q with only $|U_q| + |W_q|$ edges in the restructured graph, thus forming a tripartite graph G^*. Finally, in Line 21, any remaining edges from G that were not part of an extracted δ-clique (often considered as trivial cliques) are incorporated into the restructured tripartite graph G^*. The algorithm then returns

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

the complete restructured graph G^* in Line 22.

Time Complexity of Seq-RCP. To determine the execution time of Seq-RCP we use the same approach as in [1]. We first find the number of edges that are removed by Seq-RCP in each iteration of the while loop in Lines 5-19. There are $\gamma = n/\hat{k}$ δ-cliques that are removed. Considering $|U_q| = n^{1-\delta}$ and $|W_q| = \hat{k}$, in each δ-clique, Seq-RCP removes $\hat{k}n^{1-\delta}$ edges, therefore for γ δ-cliques Seq-RCP removes $n^{2-\delta}$ edges. The execution time to remove $n^{2-\delta}$ edges is determined by the while loop in Lines 7-18 which is dominated by the running time of Lines 9-12 and Lines 13 and 15. The for loop in Lines 9-12 takes $O(\hat{k})$ time to select randomly the vertices from the right partition. Finding the common neighbors of W_q to form a δ-clique (Line 13), and updating the adjacency matrix A (Line 15) take $O(n\hat{k})$ time. The while loop in Line 7 executes $O\left(\left\lceil \frac{n}{\hat{k}} \right\rceil\right)$ iterations. Therefore, the execution time to remove $n^{2-\delta}$ edges is $O(n^2)$. On the average it takes $\frac{n^2}{n^{2-\delta}} = O(n^\delta)$ time to remove one edge. Thus, to remove m edges it takes $O(mn^\delta)$ time on the average. The restructuring of the graph (Lines 20-21) takes $O(m)$. Therefore, Seq-RCP runs in $O(mn^\delta)$ time on the average.

4.2 Parallel Randomized Clique Partitioning-Based Algorithm for Graph Restructuring (Par-RCP)

In this section, we present our Parallel Randomized Clique Partitioning-Based Algorithm for Graph Restructuring (Par-RCP). Designed for an EREW (Exclusive Read, Exclusive Write) PRAM model, Par-RCP leverages parallelism to significantly accelerate the graph restructuring process described in Section 4.1.

Similar to its sequential counterpart, Par-RCP obtains a restructured graph G^* from an input graph $G = (U, W, E)$ by partitioning G into edge-disjoint bipartite cliques and subsequently replacing each clique with a new vertex connected to its respective left and right partitions. However, unlike the sequential approach, Par-RCP is designed to identify and extract multiple distinct bipartite cliques concurrently within each iteration. This parallel extraction continues until no further cliques can be efficiently found to reduce the edge count in G^*. The output restructured graph G^* maintains the property that each edge in the original graph G belongs to exactly one clique, i.e., $E(C_i) \cap E(C_j) = \emptyset$ for all $i \neq j$. Par-RCP is formally presented in Algorithm 2. Its input consists of the adjacency matrix A, the number of vertices n, the number of edges m in the given graph G, and a constant δ such that $0 \leq \delta \leq 1$.

Similar to its sequential counterpart, the core of Par-RCP is built upon three main phases: vertex selection, clique extraction, and graph restructuring. While performing the same fundamental task of transforming an input graph G into a restructured graph G^* by extracting and replacing δ-cliques with a tripartite representation, Par-RCP's primary objective is to achieve this process significantly faster through parallel execution. It adheres to all the restructuring properties discussed in the sequential algorithm section, notably the disjointness of extracted cliques and the preservation of path information, but executes these operations concurrently. Par-RCP is presented in Algorithm 2. Its input

Algorithm 2 Par-RCP: Parallel Randomized Clique Partitioning-Based Algorithm for Graph Restructuring

Input: A: adjacency matrix of $G = (U, W, E)$
n: number of vertices of G, $n = |W| = |U|$
m: number of edges of G.
δ: constant, such that, $0 \leq \delta \leq 1$.

1: $\mathcal{C} \leftarrow \emptyset$
2: $q \leftarrow 0$
3: $\hat{m} \leftarrow m$
4: $\hat{k} \leftarrow \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$
5: **while** $\hat{k} > 1$ **do**
6: $q \leftarrow q + 1$
7: $\gamma \leftarrow \lceil n/\hat{k} \rceil$
8: $\Psi \leftarrow [0]_n$
9: $\pi_s \leftarrow \emptyset \ \forall s \in [1, \ldots, \gamma]$
10: $U_{q \cdot s} \leftarrow \emptyset \ \forall s \in [1, \ldots, \gamma]$
11: $\tau \leftarrow \gamma$
12: **while** $\tau > 0$ **do**
13: **for all** $i \in [1, \ldots, n]$ **pardo**
14: **if** $\Psi_i \leq 0$ **then**
15: Draw s randomly from $[1, \ldots, \tau]$
16: $\pi_s \leftarrow \pi_s \cup \{i\}$
17: **for all** $s \in [1, \ldots, \tau]$ **pardo**
18: $W_{q \cdot s} \leftarrow \emptyset$
19: $Q_s \leftarrow 0$
20: **if** $|\pi_s| \geq \hat{k}$ **or** $\tau \leq 1$ **then**
21: $W_{q \cdot s} \leftarrow$ first min($\hat{k}$, $|\pi_s|$) elements of π_s
22: $Q_s \leftarrow 1$
23: **for** $i \in W_{q \cdot s}$ **do**
24: $\Psi_i \leftarrow 1$
25: Update $\tau = \tau - Q_s$ using reduction
26: **for all** $u, w \in |U| \times |W|$ **pardo**
27: **if** $\prod_{w \in W_{q \cdot s}} A_{u,w} \neq 0$ **then**
28: $U_{q \cdot s} \leftarrow U_{q \cdot s} \bigcup \{u\}$
29: **for all** $s \in [1, \ldots, \gamma]$ **pardo**
30: $C_{q \cdot s} \leftarrow \{U_{q \cdot s}, W_{q \cdot s}\}$
31: $e_{q \cdot s} \leftarrow |U_{q \cdot s}| \times |W_{q \cdot s}|$
32: Update $\hat{m} = \hat{m} - e_{q \cdot s}$ using reduction
33: $\hat{k} \leftarrow \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$
34: For each clique in $\mathcal{C} = \{C_1, \ldots, C_{q \cdot \gamma}\}$ add edges from all the vertices in the left partition U_q to an additional vertex z_q, and from z_q to all the vertices in the right partition W_q. This would restructure the graph by replacing $|U_q| \times |W_q|$ edges with $|U_q| + |W_q|$ edges.
35: The remaining edges in G are trivial cliques and are added in the restructured graph G^*.
36: **return** Restructured Graph G^*

consists of the adjacency matrix A, the number of vertices n, the number of edges m in the given graph G, and a constant δ such that $0 \leq \delta \leq 1$.

Initialization. In Lines 1-4 of Algorithm 2, Par-RCP initializes global variables before the main loop. These include setting the collection of extracted bipartite cliques $\mathcal{C}$ to an empty set, initializing the clique index q to 0, and setting the count of remaining edges in G, denoted as $\hat{m}$, to the initial total number of edges m. It also computes the initial size of the right partition for the δ-cliques, $\hat{k}$, which is a function of n, $\hat{m}$, and δ, guaranteeing the existence of a δ-clique as defined in Feder and Motwani [1].

In each iteration of the main while loop (Lines 5-33), Par-RCP initializes additional variables essential for parallel

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

clique formation. These include γ, which specifies the number of δ-cliques to be formed in the current parallel iteration, its value depending on the updated $\hat{k}$ and the number of vertices n in G. The algorithm also initializes Ψ, an array of size n (Line 8), used to keep track of vertices already selected for any δ-clique within the current iteration. Furthermore, π_s and $U_{q\cdot s}$ are initialized to empty sets and store the selected vertices for the right partition of a δ-clique s and the common neighbors forming its left partition, respectively (Lines 9-10). Finally, τ, initialized to γ in Line 11, tracks how many δ-cliques are yet to be formed within the current parallel iteration.

Vertex Selection. For Par-RCP, the primary challenge in vertex selection for parallel execution lies in ensuring that the right partitions of the δ-cliques selected in a single iteration of the main loop (Line 5) remain mutually disjoint. That is, $W_i \cap W_j = \emptyset$ for $i \neq j$, where W_i and W_j are the right partitions of δ-cliques indexed by i and j, respectively ($i, j \in [1, \gamma]$). This disjointness is critical for correct parallel processing. To address this challenge and facilitate parallel selection, the while loop in Lines 12-25 performs the vertex selection process in two parts.

First, it distributes vertices $w_i \in W$ to the γ right partitions $W_{q\cdot s}$ of the δ-cliques ($s \in Q$, where $Q = [1, \ldots, \tau]$) with a uniform probability of $1/\gamma$. To perform this distribution, for every processor i with $\Psi_i \leq 0$ (Lines 13-16), the algorithm randomly selects an index s from $[1, \tau]$ with a uniform probability of $1/\gamma$, and then the vertex corresponding to processor i is added to the set π_s. This method efficiently assigns available processors to γ δ-cliques, simultaneously selecting vertices for their respective right partitions.

Second, after these τ π_s sets are formed with the indices of potential vertices for the right partitions, the for loop in Lines 17-24 forms the actual right partitions of the δ-cliques. For each π_s that has at least $\hat{k}$ vertices, the first $\hat{k}$ vertices are assigned to form the right partition $W_{q\cdot s}$ for the δ-clique indexed $q \cdot s$. Subsequently, Q_s is set to 1 to indicate that the right partition for this clique has been successfully formed. Additionally, Ψ_i is set to 1 for all vertices i in $W_{q\cdot s}$, signifying that these vertices have been successfully assigned to a δ-clique and should not be considered for reselection in another clique during the current iteration. Finally, in Line 25, τ is updated to reflect how many more δ-cliques can be formed in the current iteration of the main while loop on Line 5. Note that the if condition on Line 20 also checks if $\tau \leq 1$, which ensures that if only one δ-clique is left to form, the algorithm will select all remaining available vertices to constitute its right partition.

Clique Extraction. The for loop in Lines 26-28 finds in parallel the common neighbours for the right partition $W_{q\cdot s}$ $\forall s \in [1, \ldots, \gamma]$. To find the common neighbours, the if condition in Line 27 checks the product of $A_{u,w}$, $\forall w \in W_{q\cdot s}$. If the condition is *true* then there exists an edge between u and $\forall w \in W_{q\cdot s}$, therefore in Line 28 the algorithm adds u to the left partition $U_{q\cdot s}$ of the $q \cdot s$ δ-clique. When all the common neighbours are found then the algorithm forms in parallel the δ-cliques $C_{q\cdot s} = \{U_{q\cdot s}, W_{q\cdot s}\}$ (Line 30) and finds the number of edges removed $e_{q\cdot s} = |U_{q\cdot s}| \times |W_{q\cdot s}|$ by each extracted δ-clique (Line 31). In Line 32, the algorithm employs a reduction operation to update $\hat{m}$ by subtracting the edges removed by the δ-cliques. In Line 33, $\hat{k}$ is updated using the updated $\hat{m}$ and the while loop in Line 5 continues until $\hat{k} > 1$.

Graph Restructuring. When the while loop (Lines 5-33) terminates, Par-RCP constructs the restructured graph G^* by adding a new vertex set Z to the bipartite graph. Each vertex of the left and right partitions of clique C_q are connected to a vertex z_q via an edge, thus forming a tripartite graph. This decreases the number of edges from $|U_q| \times |W_q|$ to $|U_q| + |W_q|$, thus restructuring the graph (Line 34). The remaining edges in the given bipartite graph $G(U, W, E)$ connecting the set U and W, which are not part of any bipartite *δ-clique* are connected directly in the tripartite graph (Line 35). Therefore, no edge is skipped during the construction of G^*, and thus, the restructured graph obtained by the Par-RCP preserves the path information of the original graph (Line 36).

Time Complexity of Par-RCP. To obtain the execution time of Par-RCP, we follow the same approach as Seq-RCP. Par-RCP removes $\Omega(n^{2-\delta})$ edges in one execution of the outer while loop in Lines 5–33. The execution time of this loop depends on the vertex-distribution loop in Lines 12–25 and the common-neighbor construction in Lines 26–31. The while loop in Lines 12–25 executes until the vertices $w_j \in W$ are assigned to the right partitions. The active τ sets are not required to receive at least $\hat{k}$ vertices in the same round. Instead, the sets with at least $\hat{k}$ vertices are finalized, while the remaining unselected vertices are redistributed among the unfinished active τ sets in subsequent rounds. For the analysis below, we assume that $\hat{k}$ divides n and let $\gamma = n/\hat{k}$ denote the number of full partitions. If $\hat{k}$ does not divide n, the remaining fewer than $\hat{k}$ vertices form one residual partition and are handled separately. This does not affect the asymptotic bound.

Lemma 1. *Suppose that τ sets remain active at the beginning of a Par-RCP distribution round and that $\tau\hat{k}$ vertices remain unselected. Each unselected vertex is assigned independently and uniformly to one of the τ active sets. Let X_s denote the number of vertices assigned to an active set π_s. Then $\mathbb{E}[X_s] = \hat{k}$. Furthermore, for every $0 < \epsilon < 1$, $P\left(\left|X_s - \hat{k}\right| \geq \epsilon\hat{k}\right) \leq 2\exp\left(-\frac{\epsilon^2\hat{k}}{3}\right)$, and $P(X_s \geq \hat{k}) \geq \frac{1}{2}$. If T denotes the number of distribution rounds required to finalize all γ full sets, then $P(T > r) \leq \gamma 2^{-r}$. Therefore, for a dense input graph G, all full sets are finalized within $O(\log n)$ rounds with high probability (w.h.p.).*

Proof. The probability that an unselected vertex is assigned to a particular active set π_s is $1/\tau$. Since $\tau\hat{k}$ vertices are distributed independently during the round, $X_s \sim \text{Binomial}\left(\tau\hat{k}, \frac{1}{\tau}\right)$. Therefore, $\mathbb{E}[X_s] = \tau\hat{k} \cdot \frac{1}{\tau} = \hat{k}$.

Since X_s is the sum of $\tau\hat{k}$ independent Bernoulli random variables, the Chernoff bound gives, for every $0 < \epsilon < 1$, $P\left(\left|X_s - \hat{k}\right| \geq \epsilon\hat{k}\right) \leq 2\exp\left(-\frac{\epsilon^2\hat{k}}{3}\right)$. This bound shows that the occupancy of a fixed set is concentrated around $\hat{k}$. However, concentration around $\hat{k}$ does not by itself imply that the set is finalized, which requires $X_s \geq \hat{k}$.

A standard median bound for a binomial random variable states that a median lies between the floor and ceiling of its mean. Since the mean of X_s is the integer $\hat{k}$, $\hat{k}$ is a median

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

of X_s. Therefore, $P(X_s \geq \hat{k}) \geq \frac{1}{2}$. A set is finalized whenever it receives at least $\hat{k}$ vertices. Exactly $\hat{k}$ of those vertices are selected for the corresponding right partition, while any excess vertices remain unselected and are redistributed in the next round. Conditioned on π_s being active at the beginning of any round, the probability that it remains active after that round is at most $1/2$. Applying this conditional bound successively gives $P(\pi_s \text{ remains active after } r \text{ rounds}) \leq 2^{-r}$. Applying a union bound over the initially active γ full sets gives

$$P(T > r) = P(\text{at least one set remains active after } r \text{ rounds})$$
$$\leq \sum_{s=1}^{\gamma} P(\pi_s \text{ remains active after } r \text{ rounds}) \leq \gamma 2^{-r}.$$

For any constant $c > 0$, choosing $r = \lceil \log_2 \gamma + c \log_2 n \rceil$ gives $P(T > r) \leq \gamma 2^{-(\log_2 \gamma + c \log_2 n)} = n^{-c}$. Since $\hat{k} = O(\log n)$ and $\gamma = n/\hat{k}$ in the dense-graph setting, $\log \gamma = \Theta(\log n)$. Therefore, all full sets are finalized within $O(\log n)$ rounds w.h.p. □

Theorem 1. *For dense graphs, Par-RCP runs on the EREW PRAM in $O(n^{\delta} \log^2 n)$ time using $O(n^2)$ processors and has a cost of $O(n^{2+\delta} \log^2 n)$ w.h.p.*

Proof. By Lemma 1, the vertex-distribution phase in Lines 12–25 requires $O(\log n)$ rounds w.h.p. In each distribution round, every unselected vertex writes its selected active-set index to a private array location. To construct the sets π_s without concurrent writes, the assignments are represented by an indicator array $\mathcal{I}$, where $\mathcal{I}[i, s] = 1$ if vertex w_i selects set π_s, and 0, otherwise. Parallel prefix sums are performed independently on the columns of $\mathcal{I}$. These prefix sums count the vertices assigned to each set and assign every vertex a unique position within its selected set. Since the number of vertex–set pairs is at most $n\gamma \leq n^2$, these operations can be performed using $O(n^2)$ processors while satisfying the exclusive-write requirement.

The sets satisfying $|\pi_s| \geq \hat{k}$ are identified in parallel. Exactly $\hat{k}$ vertices are selected from each completed set, and prefix sums assign unique locations for updating the selection array Ψ. The number of completed sets is computed using a binary-tree reduction, after which τ is updated. These operations require $O(\log n)$ time in one distribution round. Since Lemma 1 bounds the number of distribution rounds by $O(\log n)$ w.h.p., the total running time of the distribution phase is $O(\log^2 n)$ w.h.p. The common-neighbor phase in Lines 26–28 is performed in parallel for all pairs consisting of a vertex u_i and a right partition $W_{q,s}$. Since the right partitions are disjoint, each adjacency entry associated with an original edge is examined by only one vertex–partition computation. For every pair $(u_i, W_{q,s})$, a binary-tree AND reduction over the $\hat{k}$ corresponding adjacency entries determines whether u_i is adjacent to every vertex in $W_{q,s}$. This operation takes $O(\log \hat{k})$ time.

An indicator is produced for every vertex satisfying the common-neighbor condition. Parallel prefix sums over these indicators assign unique positions to the qualifying vertices in $U_{q,s}$, thereby avoiding concurrent writes during the construction of the common-neighbor sets. The updates to $\hat{m}$ and the number of removed edges are performed using binary-tree reductions and require $O(\log n)$ time. To generate G^*, the algorithm first computes the number of output items contributed by each extracted bi-clique and by the retained original edges. Parallel prefix sums then assign disjoint output offsets to the auxiliary vertices, retained original edges, and biclique-incidence edges. Each processor writes to a unique output location. Therefore, the output-generation step satisfies the exclusive-read and exclusive-write requirements of the EREW PRAM model and takes $O(\log n)$ time. The vertex-distribution phase dominates the other operations. Consequently, one execution of the outer while loop in Lines 5–33 takes $O(\log^2 n)$ time w.h.p.

Using the same edge-removal analysis as Seq-RCP, under the density assumptions used in the edge-reduction analysis, one execution of the outer while loop removes $\Omega(n^{2-\delta})$ edges. Therefore, the number of outer iterations required to process m edges is $O\left(\frac{m}{n^{2-\delta}}\right)$. Since one outer iteration takes $O(\log^2 n)$ time w.h.p., the total running time is $O\left(\frac{m \log^2 n}{n^{2-\delta}}\right)$ w.h.p. For dense graphs, where $m = \Theta(n^2)$, this becomes $O(n^{\delta} \log^2 n)$ w.h.p. Since Par-RCP uses $O(n^2)$ processors, its cost is $O(n^{2+\delta} \log^2 n)$ w.h.p. □

Relationship between the theoretical and practical models. The EREW PRAM analysis characterizes the asymptotic parallel depth of Par-RCP under an idealized shared-memory model using up to $O(n^2)$ processors. Our MPI implementation is a practical realization using $p \ll n^2$ processes on a multicore node and does not claim to attain the PRAM bound directly. The practical execution time additionally includes communication, synchronization, load imbalance, memory contention, and other hardware-dependent overheads. Accordingly, the EREW PRAM analysis describes the theoretical parallelism of the algorithm, while the MPI experiments measure how this parallelism is realized on real hardware.

5 Properties of the Proposed Algorithms

In this section, we describe and prove the properties of our Seq-RCP and Par-RCP algorithms.

5.1 Path information

We first show that the restructured graph $G^*(U, W, Z, E^*)$ obtained by Seq-RCP and Par-RCP preserves the path information of the original graph $G(U, W, E)$.

Theorem 2. *The restructured graph $G^*(U, W, Z, E^*)$ obtained by Seq-RCP and Par-RCP preserves the path information of the original graph $G(U, W, E)$.*

Proof. To show that the path information of $G(U, W, E)$ is preserved in the restructured graph $G^*(U, W, Z, E^*)$, we first deconstruct G^* to obtain the given bipartite graph $B(U', W', E')$, and then show that G and B are isomorphic. To deconstruct B we reverse the actions of the Seq-RCP or Par-RCP algorithms and replace each pair of edges (u, z_q) and (z_q, w) with an edge (u, w), for all $u \in U$, $w \in W$, $z_q \in Z$. That is, we replace all the paths (u_i, z_q, w_j) for all $u_i \in U, z_q \in Z$ and $w_j \in W$ with the edge (u_i, w_j) in B. This will replace $|U_q| + |W_q|$ edges in G^* with $|U_q| \times |W_q|$ edges in B. We do not replace the

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

trivial edges $e_{u_i,w_j} \in \hat{E}$ since they directly connect vertices from $u_i \in U$ to $w_j \in W$ in B. Therefore, the set of edges in B and G are the same i.e., $E' = E$. Also, the left and right partitions of graphs B and G are the same as those of G^*. Thus, graphs B and G are isomorphic, which is only possible when the restructured graph G^* preserves the path information of the given graph G. □

5.2 Reduction in the Number of Edges

We quantify the reduction in the number of edges achieved by our sequential and parallel randomized algorithms as the ratio $\frac{m}{m^*}$, where m and m^* are the number of edges in the given and restructured graphs, respectively. A higher value indicates a greater reduction in the number of edges. We also define the density of a graph as $\rho = \frac{|E_G|}{|E_{K_{|G|}}|}$, which is the ratio of the number of edges in graph G over the number of edges of its corresponding complete graph. Considering G as a bipartite graph, its density is $\rho(U,W) = \frac{|E(U,W)|}{|U|\times|W|}$. Then, we demonstrate the existence of a *δ-clique* in a bipartite graph G with density ρ. Following this, we determine the bound on the reduction in the number of edges achieved by the extraction of such δ-cliques using Seq-RCP and Par-RCP.

Theorem 3. *There exists a* δ-clique $C_q = (U_q, W_q)$ *with* $|U_q| \geq 1$ *and* $|W_q| = \hat{k} = k(m,n,\delta)$ *in a bipartite subgraph* $H(U', W_q, E') \subseteq G(U,W,E)$ *with density* $\rho > 1 - \frac{1}{\hat{k}}$.

Proof. We prove this theorem using the pigeonhole principle. We consider that the number of pigeons is the same as the number of edges in H i.e., $|E'| = (1 - 1/\hat{k})|U'|\hat{k} + 1$ pigeons. Thus the density ρ of graph $H > (1 - 1/\hat{k})$. If each pigeon in the set of $|E'|$ pigeons is placed in one hole among $|U'|\hat{k}$ holes arranged in $|U'|$ rows and $\hat{k}$ columns, then there are $b = |U'|\hat{k} - |E'|$ empty holes left. Solving the equation by substituting $|E'|$ and ρ we obtain,

$$b = |U'|\hat{k} - ((1 - 1/\hat{k})|U'|\hat{k} + 1) = |U'| - 1$$

which implies that, there is at least one row of holes with $\hat{k}$ pigeons. Consequently, we say that there is at least one common neighbour for W_q, and thus, there exists a *δ-clique* $C_q = (U_q, W_q)$ with $|U_q| \geq 1$ and $|W_q| = \hat{k}$ when $\rho > 1 - \frac{1}{\hat{k}}$ proving the theorem. □

We can say that the size of the left partition of the *δ-clique* C_q is at least $|U_q| \geq \rho|U'|\hat{k} - |U'|(\hat{k} - 1)$. We can easily check Theorem 3 for the case of maximum density, i.e., $\rho = 1$. In this case, for a complete bipartite graph, the size of the left partition of a *δ-clique* C_q with $|W_q| = \hat{k}$ is $|U'| = |U| = n$.

Lemma 2. *Let* $G(U,W,E)$ *be a bipartite graph with density* ρ, *where* $|U| = |W| = n$. *Let* $H(U', W', E')$ *be the subgraph of* G, *where* $W' \subseteq W$ *with* $\hat{k} = k(n,m,\delta)$ *vertices selected uniformly at random,* $U' \subseteq U$ *to consist of all vertices in* U *that have at least one edge to vertices in* W' *and* $E' \subset E$ *consists of edges between* U' *and* W'. *Then, w.h.p. , the density* $d(H)$ *of the subgraph* $H(U', W', E')$, *satisfies:* $d(H) \approx \rho$ *where* $d(H) = \frac{|E'|}{|U'|\times|W'|}$.

Proof. To prove the lemma, we analyze the expected number of edges in the subgraph H. The expected number of edges $|E'|$ in H is $\mathbb{E}[|E'|] = \sum_{u\in U}\sum_{w\in W'}\mathbb{E}[X_{u,w}]$ where $X_{u,w}$ is 1 if edge (u,w) exists and 0 otherwise. Since the selection of vertices in W' is uniform, we have, $\mathbb{E}[X_{u,w}] = \frac{\hat{k}}{n}\mathbb{P}[(u,w) \in E]$. Thus, $\mathbb{E}[|E'|] = \frac{\hat{k}}{n}|E| = \frac{\hat{k}}{n} \cdot \rho n^2 = \rho n\hat{k}$. We then apply the Chernoff bounds to show the concentration around this expected value. Let $X = |E'| = \sum_{u\in U}\sum_{w\in W} X_{u,w}$. Then, $\mathbb{P}(|X - \mathbb{E}[X]| \geq \epsilon\mathbb{E}[X]) \leq 2\exp(-\epsilon^2\mathbb{E}[X]/3)$ for $0 < \epsilon < 1$. Thus, $d(H) = \frac{|E'|}{|U'|\times|W'|} \approx \frac{|E|}{|U|\times|W|} = \rho$ whp. □

Theorem 4. *The reduction in the number of edges achieved when a bipartite graph* $G(U,W,E)$ *with density* ρ *is restructured by extracting a* δ-clique *and obtaining a tripartite graph* $G^*(U,W,Z,E^*)$, *is bounded by* $O\left(\frac{\rho n\hat{k}}{\rho n+\hat{k}}\right)$, *where* $\rho > 1 - \frac{1}{\hat{k}}$.

Proof. To prove the bound on the reduction in the number of edges when extracting a single *δ-clique*, we assume a complete bipartite graph G (i.e., $\rho = 1$), as this scenario leads to the maximum possible edge reduction for a given δ-clique size. Thus, from Lemma 2, the subgraph $H(U', W', E')$ of G, with $|W'| = \hat{k} = k(n,m,\delta)$, $U' \subseteq U$ and $E' \subset E$, has density $d(H) \approx \rho$.

Therefore, the *δ-clique* $C_q = \{U_q, W_q\}$ extracted from the subgraph H has $|U_q| = |U'|$ and $|W_q| = |W'|$. For a complete bipartite graph (H is effectively $K_{|U'|,|W'|}$ with density $\rho = 1$ for this part of the argument), the number of edges replaced is $e_{old} = |U_q|\times|W_q| = |U'|\times\hat{k}$. The number of new edges after restructuring is $e_{new} = |U_q|+|W_q| = |U'|+\hat{k}$ (by adding an additional vertex $z_q \in Z$). In the case where the δ-clique essentially is the entire graph being restructured (as implied by no remaining edges in H that are not part of C_q), the initial edges are $m = \rho n\hat{k}$ and the new edges are $m^* = \rho n + \hat{k}$.

Thus, the achieved reduction in the number of edges for the *δ-clique* C_q is given by the ratio of old edges to new edges, which is $O\left(\frac{\rho n\hat{k}}{\rho n+\hat{k}}\right)$. Furthermore, if γ such *δ-cliques* are extracted, and assuming they are edge-disjoint as per the algorithm's design, the overall edge reduction is $O\left(\frac{\gamma\rho n\hat{k}}{\gamma(\rho n+\hat{k})}\right) = O\left(\frac{\rho n\hat{k}}{\rho n+\hat{k}}\right)$. □

Theorem 5. *Given a bipartite subgraph* $H(U', W_q, E') \subset G$, *with density* $\rho > 1 - \frac{1}{\hat{k}}$, *and* $|W_q| = \hat{k} = k(m,n,\delta)$, *the reduction in the number of edges,* ER, *achieved by restructuring* H *into a tripartite graph* $H^*(U', W_q, Z_q, E^*)$ *by extracting a* δ*-clique is* $O(\log n)$. *Here,* $ER = \frac{|E'|}{|E^*|}$.

Proof. According to Theorem 3, there exists a *δ-clique* C_q with right partition U_q of size $|U_q| \geq 1$. The tripartite graph H^* is thus formed by substituting $|U_q| \times |W_q|$ edges in H with $|U_q| + |W_q|$ edges, along with the trivial edges, $|\hat{E}| = \rho|U'|\hat{k} - |U_q|\hat{k}$, that are not part of the δ-clique C_q. Therefore, the reduction in the number of edges, ER, is $ER = \frac{\rho|U'|\hat{k}}{(|U_q|+\hat{k})+|\hat{E}|}$. As a consequence of Theorem 4, the number of remaining edges, $|\hat{E}|$, depends on the density ρ. As ρ increases from $1 - \left(\frac{1}{\hat{k}}\right)$ to 1, the number of common neighbors in the right partition, W_q, increases, leading to a decrease in the number of remaining edges $\hat{E}$. In the proof of Theorem 4 it is shown that $|\hat{E}| = 0$ when $\rho = 1$. Hence,

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

we conclude that $|\hat{E}|$ decreases as ρ increases. Therefore, we subtract $|\hat{E}|$ from the numerator and denominator, which results in: $ER \leq \frac{\rho|U'|\hat{k}-|\hat{E}|}{(|U_q|+\hat{k})+|\hat{E}|-|\hat{E}|} = \frac{\rho|U'|\hat{k}-(\rho|U'|\hat{k}-|U_q|\hat{k})}{|U_q|+\hat{k}}$ Thus, $ER \leq \frac{|U_q|\hat{k}}{|U_q|+\hat{k}}$. Since $\hat{k} \approx \log n$ and $|U_q| = n^{1-\delta}$, we have $ER = O(\log n)$. □

We now show that the bound on the reduction of the number of edges obtained by our algorithms, Seq-RCP and Par-RCP, is $ER = O(\log n)$.

Theorem 6. *Under the density assumptions of Theorem 5, the reduction in the number of edges achieved by* Seq-RCP *and* Par-RCP *algorithms is* $ER = O(\log n)$.

Proof. From Theorem 5, ER for restructuring a subgraph H of G is $O(\log n)$. Algorithms Seq-RCP and Par-RCP form λ subgraphs from G to extract λ δ-*cliques*, where λ is the number of δ-cliques that the algorithms find until $\hat{k} > 1$. Therefore, ER obtained from restructuring H_q to H_q^*, $\forall q \in [1, \lambda]$ is, $ER = \frac{\lambda\rho n\hat{k}}{\lambda((\rho\hat{k}+\hat{k})+|\hat{E}|)}$ which results in $O(\log n)$ as shown in Theorem 5. Thus, Algorithms Seq-RCP and Par-RCP achieve a reduction in the number of edges $ER = O(\log n)$. □

It is worth noting that the bound on $ER = O(\log n)$ is a conditional guarantee under the density assumption $\rho > 1 - 1/\hat{k}$ used in the analysis. Under the same density condition, this bound would also apply to the FM algorithm [1]. However, in FM $\hat{k} = k(m, n, \delta)$ is updated after extracting each δ-*clique*, which results in lowering the value of $\hat{k}$ after every extraction of $f' \leq \gamma$ δ-*cliques*, where, $\gamma = \lfloor n/\hat{k} \rfloor$. We graphically show in Figure 2 the monotonically decreasing $\hat{k}$ in FM and thus the edge reduction obtained by FM is smaller than that obtained by Seq-CP and Par-CP. We would like to formally prove this in our future work.

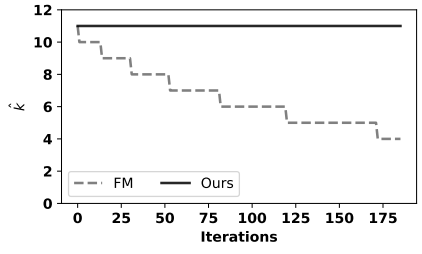


Fig. 2: Progression of $\hat{k}$ in the extraction of $\gamma = n/\hat{k}$ δ-cliques for Seq-RCP and Par-RCP, and the FM when $\delta = 1$ for a completely dense graph with approximately 2 thousand vertices and 1.04 million edges for density $= 100\%$.

5.3 Extension to non-bipartite graphs

The proposed sequential and parallel algorithms can be easily extended to restructure non-bipartite graphs using a similar technique to that described by FM [1]. Consider a directed graph $H(V, E)$ with n vertices and m edges. Initially, we transform H into a bipartite graph $G(L, R, E')$, where each vertex $v \in V$ is duplicated into $v_L \in L$ and $v_R \in R$. For each directed edge (u, v) in E, we create a directed edge (u_L, v_R) in E', with the direction from L to R. Furthermore, for each vertex $v \in V$ we add a directed edge from v_R to v_L. The proposed restructuring algorithms are then applied to the bipartite graph G, which includes only the edges from L to R. Once the restructured graph is computed we add to it the n directed edges from u_R to u_L, corresponding to each vertex $u \in V$. Since the path information of the original graph is preserved by this transformation, it allows different graph algorithms to work on the restructured graph as well. Undirected general graphs can be transformed first into directed graphs by simply replacing each undirected edge (u, v) in the original graph by two directed edges (u, v) and (v, u), and then applying the technique presented above to transform the directed graph into a bipartite graph. The time complexity and the edge reduction of the proposed algorithms are the same as in the case of the bipartite graphs.

The extension to non-bipartite graphs also introduces additional storage. For a general graph $H = (V, E)$ with $|V| = n$, the bipartite transformation creates two copies of each original vertex, resulting in $2n$ vertices. The restructured graph additionally contains the auxiliary vertices in Z introduced during clique replacement. Therefore, the restructured graph contains $2n + |Z|$ vertices and m^* edges. Using an adjacency-list representation, this requires $O(2n + |Z| + m^*)$ storage. Thus, the additional storage from duplicated and auxiliary vertices is balanced against the reduction in the number of edges obtained through restructuring.

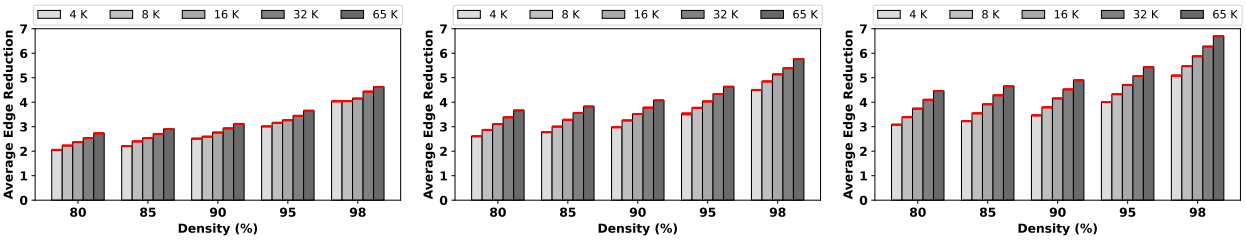
6 Experimental Results

6.1 Experimental Setup

In this section, we investigate the performance of the sequential (Seq-RCP) and parallel (Par-RCP) randomized clique partitioning based graph restructuring algorithms in terms of two main performance metrics: *reduction in the number of edges* and *speedup*. We implement both algorithms in C and use the MPI library for implementing Par-RCP. The implementation is available on GitHub [22]. We run the experiments on the Regular Memory (RM) nodes of the Pittsburgh Supercomputing Center's Bridges-2 system [23]. Each RM node provides general-purpose computing and has two AMD EPYC 7742 CPUs, each CPU including 64 cores and 128 threads at a frequency between 2.25 and 3.40 GHz. It has 256GB RAM, 256MB L3 cache with 8 memory channels, and a 3.84TB NVMe SSD for local storage. We used one independent RM node with 1 CPU core for the sequential algorithm Seq-RCP, and varied the number of physical CPU cores for Par-RCP between 16 and 128, allocating one thread per core. While our experimental evaluation is conducted on a single high-performance multicore node, we deliberately implemented Par-RCP using the Message Passing Interface (MPI) rather than a shared-memory framework like OpenMP. This design choice provides strict process isolation, effectively eliminating shared-memory bottlenecks such as false sharing and race conditions—challenges that are particularly severe in irregular graph algorithms with unpredictable memory access patterns. Furthermore, modern MPI implementations highly optimize intra-node communication via zero-copy shared memory segments, ensuring that communication overhead remains minimal. Consequently, this architecture not only delivers robust single-node parallel performance but also renders the algorithmic implementation inherently prepared

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036



(a) δ= 0.6 (b) δ= 0.8 (c) δ= 1

Fig. 3: Par-RCP average ER vs. graph density for (a) δ= 0.6, (b) δ= 0.8, and (c) δ= 1, on bipartite graphs with 4 thousand (4K) to 65 thousand (65K) vertices and up to 1.05 billion edges.

for future distributed-memory scaling across multi-node clusters.

To evaluate our algorithms on large graphs, we generate both bipartite graphs and general graphs of varying densities. For bipartite graphs, we use the $G(N, p)$ random graph model, adapted for bipartite structures, where edges between the two partitions are added with probability p. General graphs are generated using the standard $G(n, p)$ random graph model, where n is the number of vertices and each possible edge is added with an independent probability p. In both models, the probability p directly measures the density of the graph, and the inclusion of each edge is independent. We implemented a Python program to generate instances of these large graphs, with the number of edges $m = |E|$ ranging from 3.3 million to 1.05 billion. We show the results for various combinations of graph density p and parameter δ, which directly controls the size of the right partition of the clique (see Section 3). Specifically, we vary the density from 80% to 98% and δ from 0.5 to 1.

Practical selection of δ. The parameter δ determines the size of the right partition through $\hat{k} = \left\lfloor \frac{\delta \log n}{\log(2n^2/m)} \right\rfloor$. As a basic feasibility check, values of δ that produce $\hat{k} \leq 1$ are excluded because they cannot lead to a nontrivial biclique replacement. For values satisfying $\hat{k} > 1$, the choice of δ depends on the density, degree distribution, and common-neighbor structure of the input graph. The experimental results below show that different values of δ can lead to different edge reductions and preprocessing times. Therefore, no single value of δ is expected to be optimal for all graphs. Developing a systematic or adaptive method for selecting δ is left for future work.

For each bipartite graph with a given density and δ, we generate 10 different instances, run each of them 5 times, and then present the average values with the standard deviation of the above metrics.

6.2 Performance Analysis on Bipartite Graphs

6.2.1 Reduction in the Number of Edges (ER).

We calculate the reduction in the number of edges (ER) of our sequential and parallel randomized algorithms as $\frac{m}{m^*}$, where m and m^* are the number of edges in the given and restructured graphs, respectively. Since the selection of $\hat{k}$ vertices from the right partition in both algorithms is random, the edge reduction achieved by Seq-RCP and Par-RCP is roughly the same. Thus, we do not show the average reduction in the number of edges for Seq-RCP and only analyze that of Par-RCP in Figures 3 and 4. Given the large number of combinations tested, in the following, we first analyze ER for fixed δ at various densities, and then analyze the results for fixed densities at different δ values. In general, we can observe that ER increases by increasing δ and the graph's density. This is because having larger δ and denser graphs results in larger $\hat{k}$ and the selection of more vertices from the right partition. In addition, having denser graphs leads to finding more common neighbors in the left partition, resulting in larger δ-cliques and consequently, a higher reduction in the number of edges.

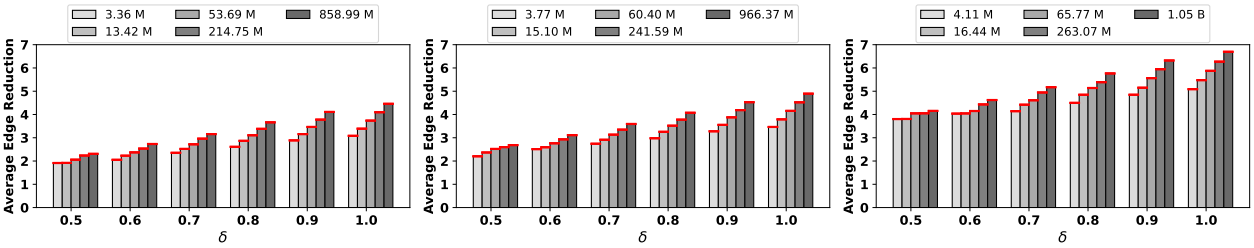
Same δ and different densities. Figures 3a, 3b, and 3c show the average ER for bipartite graphs with up to 65 thousand vertices and up to 1.05 billion edges for three δ values of 0.6, 0.8, and 1, respectively. In Figure 3a, we observe that for the same graph with 65 thousand vertices and with the same δ=0.6, progressively increasing the density from 80% to 98% leads to an average ER of 2.73, 2.9, 3.11, 3.65, and 4.62, respectively. Comparing the first and last densities translates into a 69.23% increase in the reduction of the number of edges. Increasing δ to 0.8, as Figure 3b shows, leads to a 57.38% increase in ER for graphs with 65 thousand vertices when density varies from 80% to 98%. Finally, when δ is equal to 1, as Figure 3c shows, ER increases by 50.34%. While the absolute edge reduction increases by increasing δ, at higher values of δ the algorithm can achieve a greater edge reduction at lower densities than at higher densities, leading to a smaller overall increase in ER (e.g., 69.23% to 50.34% range). This is because increasing the density translates into having more edges in the graph, resulting in larger $\hat{k}$ (i.e., the size of the right partition of the extracted cliques). Thus, having a larger $\hat{k}$ means having a larger number of vertices in the right partition of the cliques, which decreases the probability of finding common neighbors in the left partition. This, in turn, lowers the rate of increase of ER for higher densities.

However, while increasing δ may seem beneficial, as we will observe later in some cases this choice can lead to an increase in execution time due to a lower number of common neighbors considered when forming the cliques.

Same densities and different δ. Figures 4a, 4b, and 4c show the average ER for bipartite graphs with up to 65 thousand vertices and 1.05 billion edges for six different values of δ in the range [0.5,1], and densities $p = 80\%, 90\%$, and 98%, respectively. In all three figures for a graph with 65 thousand vertices and the same density, we observe an increase in the average ER as δ increases. In Figure 4a, for a graph with

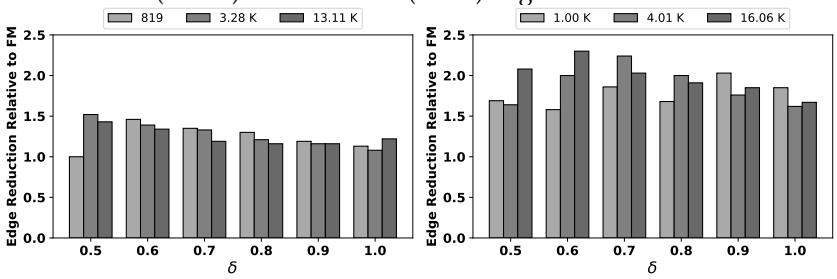
Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036



(a) 80% Density (b) 90% Density (c) 98% Density

Fig. 4: Par-RCP average ER vs. δ for densities (a) 80%, (b) 90%, and (c) 98%, on bipartite graphs with 4 thousand to 65 thousand vertices and 3.36 million (3.36M) to 1.05 billion (1.05B) edges.



(a) 80% Density (b) 98% Density

Fig. 5: Reduction in the number of edges relative to FM vs. δ: small graphs with (a) 819 to 13.11 thousand edges and 80% density; and (b) 1 thousand to 16.06 thousand edges and 98% density.

858.99 million edges with density 80%, we observe average ER of 2.3, 2.73, 3.15, 3.66, 4.1, and 4.45 for different values of delta, which represents an increase in ER by 93.48% from the case of $\delta = 0.5$ to the case of $\delta = 1$. In Figure 4b, for a graph with 966.37 million edges for density 90%, the increase in ER is about 82.46% between $\delta = 0.5$ and 1. Finally, as Figure 4c shows, for a graph with 1.05 billion edges for density 98%, the algorithm achieves an increase in ER of 61.20% by increasing the value of δ from 0.5 to 1. These results are in line with the observations made in the previous analysis.

We also observe that for the same density and δ, by increasing the number of vertices in the graph we have an increase in the number of extracted cliques, resulting in a higher ER. For example, in Figure 4b, for $\delta = 0.8$ and 90% density, the algorithm obtains an average ER of 2.97, 3.25, 3.51, 3.77, and 4.07 for graphs with 3.77 million to 966.37 million edges. This shows a maximum increase of 37.04% in ER. Finally, in almost all cases for Figure 3 and Figure 4, we observe a very small standard deviation as shown by the error bars. This is because in our experiments, the average reductions in the number of edges are consistently close to each other, with no considerable outliers.

These results show that our graph restructuring algorithms can effectively lead to high reduction in the number of edges, making them useful for speeding up the execution of graph algorithms for solving various problems in large graphs.

6.2.2 Reduction in the Number of Edges Relative to FM

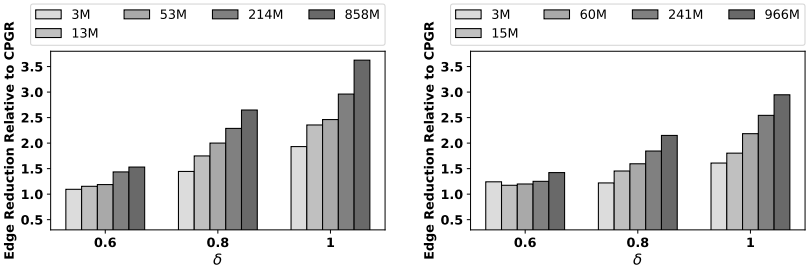
We compare the edge reduction achieved by our algorithms, Seq-RCP and Par-RCP, against that of FM on small graphs—specifically, graphs with up to 256 vertices and 16.06 thousand edges. This restriction arises from a known limitation of the FM algorithm: for larger graphs, the vertex selection process may involve extremely large integer computations that exceed standard machine precision. As a result, consistent with prior observations [2], we are only able to evaluate FM on bipartite graphs within this size range. Figure 5a and 5b show the relative reduction in the number of edges, i.e., the ratio of edge reduction obtained by our algorithms and that obtained by FM for the same small graphs with different δ, and 80% and 98% density, respectively. In all cases, except for a graph with 64 vertices and 819 edges, our algorithms obtain a higher reduction in the number of edges compared to that of FM, up to 2.3 times higher. This is mainly because our algorithms extract multiple δ-cliques in one iteration which results in removing more edges at higher values of $\hat{k}$. For the graph with 256 vertices and the same $\delta = 0.6$, increasing the density from 80% to 98% we observe an increase in the relative reduction in the number of edges by 71.64%. This is because increasing the density results in a larger $\hat{k}$ leading to a larger right partition and the probability of finding more common neighbors.

For a small graph with 64 vertices and 819 edges for 80% density, and $\delta = 0.5$ leads to the size of the right partition $\hat{k} = 1$, which does not result in a reduction in the number of edges. Thus, our algorithm and FM result in no reduction in the number of edges, hence the reduction relative to FM is 1. As δ increases, our algorithm obtains a higher reduction. We also observe that when the δ is increased to 1 the relative reduction in the number of edges decreases, this is because increasing δ increases $\hat{k}$, which lowers the probability of finding common neighbors at lower density.

We do not report runtime comparisons in this setting, as the evaluation is restricted to small graphs where runtime differences are not representative of performance on larger

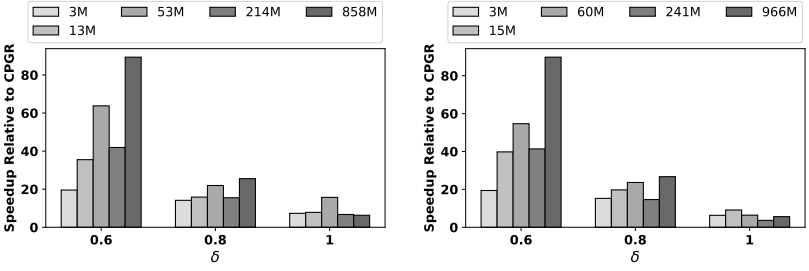
Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036



(a) 80% Density (b) 90% Density

Fig. 6: Par-RCP reduction in the number of edges relative to CPGR vs. δ: bipartite graphs with up to $\approx$ 966M edges, at (a) 80% and (b) 90% density.



(a) 80% Density (b) 90% Density

Fig. 7: Par-RCP (64 cores) speedup relative to CPGR vs. δ: graphs with up to 966M edges, at (a) 80% and (b) 90% density.

instances.

6.2.3 Reduction in the Number of Edges and Speedup Relative to CPGR

We compare the edge reduction achieved by Par-RCP against that of the state-of-the-art deterministic sequential graph restructuring algorithm CPGR [2] across a range of graph sizes and densities. Figure 6a and 6b show the relative reduction in the number of edges, i.e., the ratio of edge reduction obtained by Par-RCP to that obtained by CPGR, for varying values of δ at 80% and 98% density, respectively. Across all graph sizes, Par-RCP consistently outperforms CPGR in terms of edge reduction, with the gap widening as both the graph size and δ increase. For example, for graphs with 2048 vertices, the relative reduction ranges from $1.09\times$ to $1.93\times$, while for larger graphs with 32768 vertices, it increases from $1.53\times$ to up to $3.63\times$. This trend highlights the improved effectiveness of Par-RCP at larger scales and higher values of δ. Although both approaches extract multiple δ-cliques, CPGR restricts its search to a subgraph induced by vertices whose degree is at least $\hat{k}$. As a result, only a subset of vertices participates in clique extraction. In contrast, Par-RCP operates on the entire graph, allowing all vertices to contribute to the discovery of δ-cliques. This difference becomes more pronounced as $\hat{k}$ increases, since the degree threshold in CPGR becomes more restrictive, reducing the number of eligible vertices.

Consequently, Par-RCP is able to extract a larger number of δ-cliques, particularly at higher values of $\hat{k}$, leading to a greater reduction in the number of edges and improved compression ratios compared to CPGR.

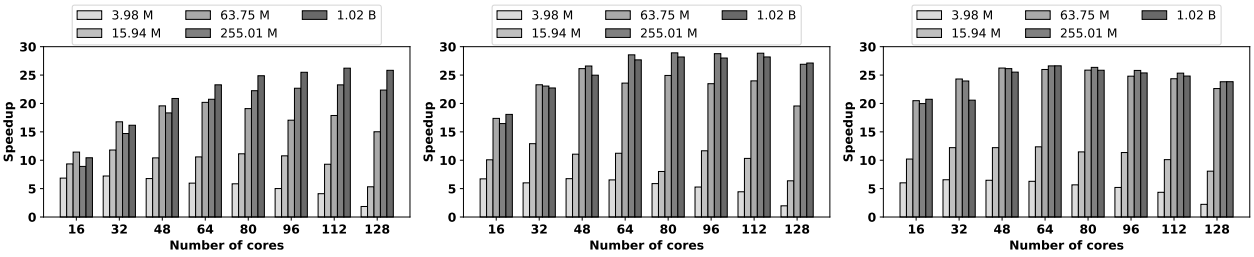
Figure 7 shows the speedup of Par-RCP relative to CPGR as a function of δ for different graph sizes and densities. Here, Par-RCP is executed using 64 cores. Overall, Par-RCP achieves significant speedups across all settings, especially at lower values of δ, where it is as high as 90. As δ increases, however, the runtime advantage gradually decreases. This behavior is largely driven by how δ affects the size of the right partition $\hat{k}$. Higher values of δ lead to larger $\hat{k}$, which makes it harder to find common neighbors. As a result, both CPGR and Par-RCP tend to terminate earlier, reducing the total runtime and bringing their performance closer. That said, Par-RCP still maintains a clear advantage. Since it operates on the entire graph, it is able to extract larger δ-cliques in each iteration, which quickly reduces the number of remaining edges and speeds up subsequent steps. In contrast, CPGR works on a restricted subset of vertices that satisfy the degree threshold, leading to a more gradual reduction of the graph. As δ increases, the chance of finding large common neighborhoods drops for both methods, which limits how much work can be done in each iteration. This is why the relative runtime improvement of Par-RCP decreases at higher values of δ, even though it remains consistently faster than CPGR.

6.2.4 Speedup Relative to Seq-RCP

We define the speedup as the ratio of the execution time of Seq-RCP and the execution time of Par-RCP. To show the performance and scalability of Par-RCP we increase the number of cores from 8 to 128 by a constant factor of 8. Figures 8a to 8c show the speedup achieved by Par-RCP

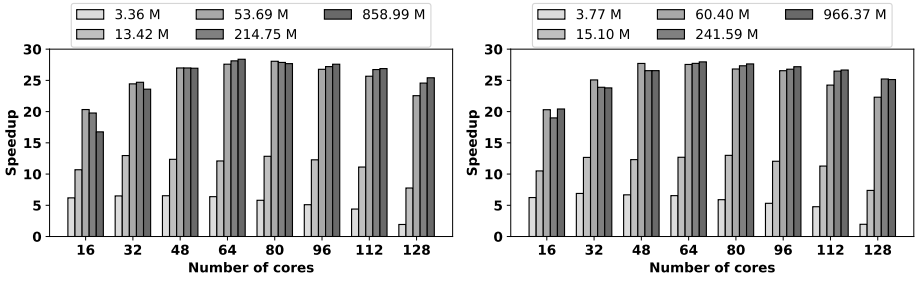
Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036



(a) δ=0.6 (b) δ=0.8 (c) δ=1

Fig. 8: Par-RCP speedup vs. number of cores for (a) $\delta = 0.6$, (b) $\delta = 0.8$, and (c) $\delta = 1$, bipartite graphs with up to 65 thousand vertices, up to 1.02 billion (1.02B) edges, and density 95%.



(a) 80% density (b) 90% density

Fig. 9: Par-RCP speedup vs. number of cores for bipartite graphs with up to 65 thousand vertices and up to 1.05 billion (1.05B) edges for different densities (a) 80% and (b) 90% with δ=0.9.

for bipartite graphs with 65 thousand vertices and up to 1.02 billion edges for density 95%, and $\delta = 0.6, 0.8$, and 1, respectively. As we can see, increasing the number of cores from 16 to 128 leads to a speedup ranging from 1.85 to 28.91. Figures 9a and 9b show the speedup with $\delta = 0.9$ for bipartite graphs with 65 thousand vertices and up to 1.05 billion edges for densities 80% and 90%, respectively. In almost all three figures, we observe a significant speedup ranging from 1.8 to 28.38 for bipartite graphs using up to 64 cores. On the other hand, in some cases, increasing the number of cores too much may lead to an increase in communication and synchronization overhead, as well as memory contention effects, that can result in a slight decrease in the achieved speedup. This is particularly noticeable for graphs with a lower number of vertices, where the overhead outweighs the benefits of fine-grained parallelism. These factors become prominent at higher core counts, even on the high-performance system described in Section 6.1, due to the inherent challenges of parallelizing irregular graph operations.

In addition, the best speedup is achieved for a different number of cores across the various graphs. For example, considering Figure 9b, for a graph with 16 thousand vertices and 60.40 million edges at 90% density, and $\delta = 0.9$, the optimal number of cores to maximize the speedup is 48. In contrast, for a graph with 65 thousand vertices (with the other parameters having the same values), the best number of cores to allocate is 64. Thus, it remains an open challenge how to choose the best combination of δ and the number of cores to allocate for a given graph ahead of time.

These results show that our parallel algorithm Par-RCP can effectively lead to substantial speedup relative to its sequential counterpart while obtaining the same high edge reduction.

6.3 Performance Analysis on General Graphs

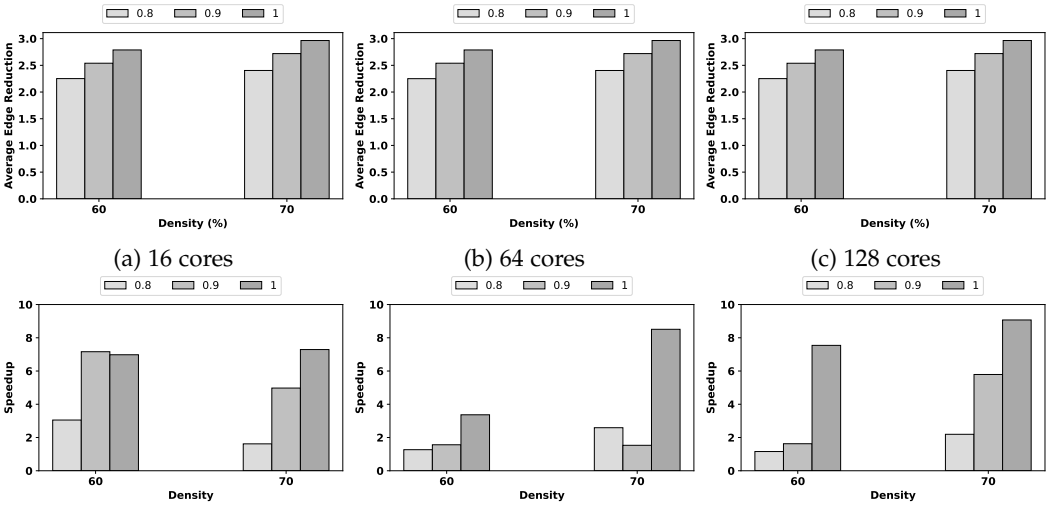
In this section, we analyze the performance of Par-RCP on general graphs with up to 32 thousand vertices. Figure 10 shows the reduction in the number of edges, ER, and speedup for graphs with different density and using different δ. These results are obtained using 16, 64, and 128 cores. This analysis is to highlight the adaptability of the algorithm on general graphs as well.

6.3.1 Reduction in the Number of Edges (ER).

Figures 10a, 10b, and 10c, illustrate the average ER achieved by Par-RCP. A key observation across all tested core counts is that ER is primarily dependent on the graph's density and the algorithm's parameter δ, and is largely independent of the number of cores used for execution. As expected, the plots for 16, 64, and 128 cores show nearly identical ER values, validating that the restructuring outcome is consistent. In all cases, ER consistently increases with increasing graph density. For instance, graphs with 70% density consistently yield higher ER compared to graphs with 60% density. Similarly, for any given density, higher values of δ result in greater ER, with $\delta = 1.0$ consistently providing the highest reduction, reaching values close to 2.95. This trend confirms that Par-RCP with higher δ, effectively reduces the number of edges in denser graphs, which is crucial for accelerating downstream graph algorithms.

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036



(a) 16 cores (b) 64 cores (c) 128 cores

(d) 16 cores (e) 64 cores (f) 128 cores

Fig. 10: Par-RCP average ER (a), (b), and (c), and speedup (d), (e), and (f) for general graphs with 32 thousand vertices, varying densities = {60%, 70%}, resulting in up to 358 million edges, and δ values ={0.8, 0.9, 1.0} with 16, 64, and 128 cores.

6.3.2 Speedup relative to Seq-RCP

Figures 10d (16 cores), 10e (64 cores), and 10f (128 cores) show the speedup achieved by Par-RCP relative to its sequential counterpart for δ values ranging from 0.8 to 1.0. We consistently observe that Par-RCP delivers good speedups, generally increasing with both increasing δ and increasing graph density. Higher δ values and denser graphs lead to more pronounced accelerations. For instance, the highest speedups across all core counts are consistently found at 70% density with $\delta = 1.0$.

As parallelism scales, the behavior becomes more intricate. While peak speedups (e.g., approximately 8.5 at 70% density, $\delta = 1.0$, representing a 13.33% increase compared to 16 cores) are observed at 64 cores (e) and approximately 9.07 at 128 cores (f), which is a 20.93% increase compared to 16 cores, the performance does not always increase monotonically across all core counts for every configuration. For instance, at 60% density and $\delta = 1.0$, speedup is approximately 7.0 at 16 cores, but then shows a decrease to approximately 3.5 at 64 cores and approximately 3.0 at 128 cores. This highlights the complex interplay of factors in parallel graph processing. Even on high-performance multicore architectures, scaling beyond an optimal point can introduce increasing communication and synchronization overheads, as well as memory contention effects, especially for irregular graph operations.

The nuanced behavior of Par-RCP's parallel speedup, observed across different parameter combinations and core counts, can be attributed to several interacting factors critical for optimal deployment. The performance of Par-RCP is profoundly influenced by the intricate interplay between the chosen δ value, graph density, and the algorithm's inherent randomness. Specifically, both δ and the graph's density directly impact the size of the right partition, $\hat{k}$, for the δ-cliques. For lower-density graphs, where the number of common neighbors (edges) is inherently smaller, a larger $\hat{k}$ can decrease the probability of quickly finding enough common neighbors. This leads to more computationally intensive common neighbor searches and can result in an uneven distribution of workload across parallel threads, where some threads are bottlenecked by the difficulty of forming valid cliques. Furthermore, the randomized nature of vertex selection means the exact computational cost of finding each δ-clique can fluctuate between runs, contributing to inconsistencies in parallel load balancing. Consequently, while Par-RCP provides substantial acceleration, maximizing its benefits requires careful consideration of the graph's characteristics and the core allocation, to ensure the gains from parallelism are not disproportionately affected by increasing overheads. Thus, the optimal balance between parallelism and overhead is highly dependent on the specific graph characteristics and the chosen value of δ.

6.4 Performance Analysis: Speeding-up Graph Algorithms

In this section, we show the speedup achieved by downstream graph algorithms when Par-RCP is used as a preprocessing step across different graph types. Specifically, we present the speedup of Dinitz's algorithm on large synthetic bipartite graphs and the speedup of the All Pairs Shortest Path (APSP) algorithm on real-world datasets from the SuiteSparse Matrix Collection [24], presented in Table 1. In addition, we evaluate APSP on low-density synthetic R-MAT graphs with skewed degree distributions, generated

TABLE 1: Real-world Datasets (SuiteSparse Matrix Collection [24])

Sr. No	Dataset name	Rows	Columns	Edges	Density
1	beacxc	497	506	50409	0.2
2	beaflw	497	507	53403	0.2
3	cari	400	1200	152800	0.3
4	mbeacxc	496	496	49920	0.2
5	mbeaflw	496	496	49920	0.2

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

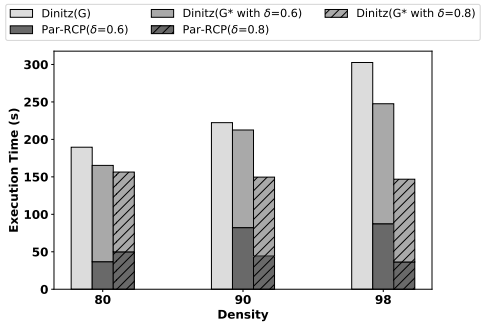


Fig. 11: Par-RCP + Dinitz(G^*) vs. Dinitz(G): Average execution time of Dinitz's algorithm on the original bipartite graph G (labeled Dinitz(G)) and on the restructured graph G^* (labeled Dinitz(G^*)) with δ = [0.6, 0.8], for a large graph with 32,764 vertices in each bipartition up to 1.05 billion edges corresponding to densities = 80%, 90%, and 98%.

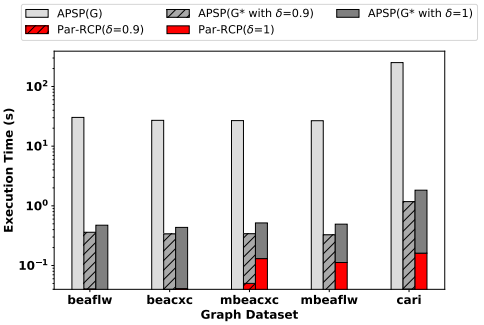


Fig. 12: Par-RCP + APSP(G^*) vs. APSP(G): Average execution time of APSP algorithm on the original bipartite graph G (labeled APSP(G)) and on the restructured graph G^* (labeled APSP(G^*)) with δ = [0.9, 1], for real-world datasets.

using NetworKit's R-MAT generator [25], with scales 14 and 15 and an edge factor of 512. Across these algorithms and graph types, using the restructured graphs with fewer edges produced by Par-RCP as input can reduce downstream execution time.

Downstream-algorithm interface. Feder and Motwani [1] showed that graph restructuring can reduce the number of edges processed by downstream graph algorithms. Following this general approach, the auxiliary vertices introduced by Par-RCP are used only to represent the extracted bicliques and are not treated as ordinary vertices of the original application graph. For each extracted biclique $C_q = (U_q, W_q)$, the two-edge segment $u - z_q - w$, where $u \in U_q$ and $w \in W_q$, represents the original edge $(u, w) \in E(G)$. For shortest-path computations, this segment is interpreted as one logical edge. During path recovery, $u - z_q - w$ is mapped to (u, w), and the auxiliary vertex z_q is excluded from the reported path. Therefore, the reported paths and distances are expressed only in terms of the original vertices and edges of G. Similarly, for maximum-cardinality bipartite matching, a unit of flow passing through $u - z_q - w$ is mapped to the matching edge (u, w). Since $U_q \times W_q$ is a complete bipartite subgraph of G, every such mapped edge is a valid edge of the original graph. The auxiliary vertices are excluded from the final matching. These mappings allow the downstream algorithms to operate directly on the restructured graph G^* without explicitly reconstructing the complete edge set of G, while ensuring that the reported outputs correspond to the original graph.

6.4.1 Speeding-up Maximum Cardinality Bipartite Matching Algorithms

Figure 11 presents a performance comparison of Dinitz's algorithm applied to the original bipartite graph G and to the restructured graph G^*, generated using Par-RCP. By preserving the path connectivity, Par-RCP enables G^* to serve as a valid input for flow-based matching algorithms like Dinitz, as well as other algorithms for maximum cardinality matching in bipartite graphs. Dinitz's algorithm [26], with a runtime of $O(\sqrt{n}m)$ for maximum cardinality bipartite matching, benefits significantly from Par-RCP preprocessing. Since Par-RCP reduces the number of edges by a factor of $k = \Omega\left(\frac{\delta \log n}{\log(2n^2/m)}\right)$, the total runtime becomes $O\left(\frac{\sqrt{n}m}{k} + n^\delta \log n\right)$ which simplifies to $O\left(\frac{\sqrt{n}m}{\log n}\right)$. This asymptotic bound matches that of state-of-the-art algorithms for maximum cardinality bipartite matching, such as the FM and push-relabel methods. Similar improvements are also expected for the Hopcroft–Karp algorithm [7]. While better asymptotic bounds have been proposed by recent algorithms like [27] (a randomized $n^{2+o(1)}$-time algorithm) and [28] (a randomized $\tilde{O}(m+n^{1.5})$-time algorithm), these approaches are largely of theoretical interest. Their practical applicability is limited due to significant implementation complexity and computational overhead.

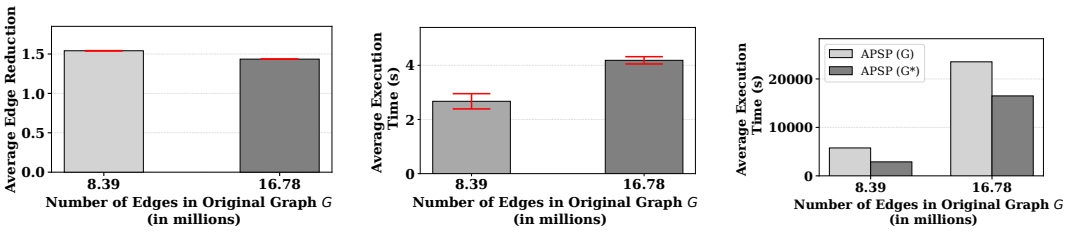
Therefore, we use Dinitz's algorithm as the baseline in our experimental evaluation. Figure 11 compares its execution time on the original bipartite graph G (with n = 32K nodes per partition and up to 1.05 billion edges, at densities of 80%, 90%, and 98%) against the combined time of Par-RCP preprocessing and Dinitz's algorithm on the restructured graph G^*. Across all cases, the combined runtime of Par-RCP + Dinitz on G^* outperforms Dinitz on G, achieving speedups of up to 2.06. On average, Par-RCP + Dinitz on G^* delivers a speedup of 1.59 relative to Dinitz on the original graph. This improvement correlates strongly with the reduction in edge count achieved by Par-RCP. For instance, at 98% density, a higher value of m and δ leads to more aggressive edge reduction, resulting in up to a 51.47% decrease in Dinitz's runtime compared to its performance on the original graph G.

6.4.2 Speeding-up All-Pairs Shortest Path (APSP) Algorithms.

Figure 12 shows the execution time of the All-Pairs Shortest Path (APSP) algorithm, implemented using Breadth-First Search (BFS), on both graph G and its restructured version G^*. For the original graphs we use five real-world sparse graphs from SuiteSparse [24] presented in Table 1. We validated correctness by confirming that the number of connected vertex pairs remains the same in both versions. On average, Par-RCP preprocessing combined with APSP on G^* achieves a speedup of 90.75 compared to APSP on the original graph G. For example, in the cari dataset, the APSP execution time decreases from 251.34s on G to 1.176s on G^* (0.04s for Par-RCP and 1.136s for APSP), resulting

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036



(a) (b) (c)

Fig. 13: Performance on the R-MAT graphs with 16K and 32K vertices and approximately 8.39M and 16.78M edges, respectively, for $\delta = 1.0$. (a) Average edge reduction achieved by Par-RCP. (b) Average execution time of Par-RCP. (c) Average execution time of APSP on the original graph G compared with the end-to-end execution time consisting of Par-RCP preprocessing followed by APSP on the restructured graph G^*. Error bars indicate the standard deviation.

in a speedup of 213.34 and a 99.53% reduction in runtime. This reduction in execution time is particularly beneficial for large-scale graph applications, where performance improvements can lead to more efficient and responsive systems.

We also observe that the execution time of APSP on G^* is lower when using $\delta = 0.9$ compared to $\delta = 1$. As discussed earlier, higher values of δ increase the size of the right partition, which in turn increases the time for finding common neighbors.

Next, we evaluate the impact of restructuring on APSP considering low-density synthetically generated R-MAT graphs. Figure 13 summarizes the evaluation of Par-RCP on the R-MAT graphs with 16K and 32K vertices for $\delta = 1.0$. The graphs contain approximately 8.39M and 16.78M edges, corresponding to densities of approximately 6.25% and 3.13%, respectively. Thus, these experiments evaluate Par-RCP on considerably sparser graphs than those used in our original synthetic evaluation.

Figure 13a shows the compression achieved by Par-RCP. The average compression ratios are approximately $1.54\times$ for the 8.39M edges graph and $1.44\times$ for the 16.78M edges graph, demonstrating that Par-RCP reduces the number of edges even for these relatively sparse R-MAT instances. The small variation across repeated executions further indicates that the restructuring behavior is stable.

Figure 13b reports the corresponding execution time. The execution time of Par-RCP remains small for both graph sizes, and it is negligible relative to the subsequent APSP computation. This is important because any benefit from operating on the restructured graph must offset the time of constructing G^*.

Figure 13c therefore compares APSP on the original graph G against the complete end-to-end execution time consisting of Par-RCP preprocessing followed by APSP on the restructured graph G^*. For the 8.39M edges graph, restructuring reduces the overall execution time by approximately 49.9%, corresponding to a speedup of about 1.99. For the 16.78M edges graph, the overall execution time is reduced by approximately 29.9%, corresponding to a speedup of about 1.43.

Overall, these results show that the reduction in graph representation achieved by Par-RCP translates into substantial downstream performance improvements. Even for the 16.78M edges graph, where the density is approximately 3.13%, Par-RCP achieves effective compression and reduces the end-to-end APSP execution time by approximately 30%. These results indicate that the benefits of restructuring are not limited to dense graphs and can also extend to large, low-density graphs with skewed degree distributions.

7 Conclusion

In this paper, we introduced Par-RCP, a randomized parallel algorithm for lossless graph restructuring based on bipartite clique partitioning. Our approach bridges the gap between theoretical clique-partitioning techniques and practical, scalable graph processing, enabling efficient restructuring of large and dense graphs while preserving complete path information. We also developed a sequential variant, Seq-RCP, which improves the theoretical running time to $O(mn^{\delta})$. Our algorithms achieve strong theoretical and practical performance. Par-RCP runs in $O(n^{\delta}\log^2 n)$ time on n^2 processors, and experimentally demonstrates high scalability on graphs with up to billions of edges. Compared to prior approaches, Par-RCP achieves significantly higher edge reduction, up to $3.63\times$ and substantial runtime improvements of up to $89.35\times$, while also delivering up to $6.69\times$ reduction in the number of edges in practice. These gains stem from its ability to operate on the entire graph and extract larger δ-cliques, enabling faster reduction of the graph structure. We further show that restructuring with Par-RCP accelerates downstream global connectivity-dependent algorithms. In particular, we observe speedups of up to 2.06 for Dinitz's maximum cardinality matching algorithm and up to 213.34 for All-Pairs Shortest Paths, demonstrating the effectiveness of our approach as a preprocessing step for large-scale graph analytics.

Despite these improvements, several challenges remain. The interplay between parallelism, graph characteristics, and the choice of parameter δ continues to influence performance. Future work includes developing adaptive strategies for selecting δ, and extending the algorithms to the case of dynamic graphs. These directions will further broaden the applicability of our approach to a wider range of graph analytics tasks.

Acknowledgement

This work used the Bridges-2 nodes at the Pittsburgh Supercomputing Center through allocation CIS230399 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036

& Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

REFERENCES

[1] T. Feder and R. Motwani, "Clique partitions, graph compression and speeding-up algorithms," *Journal of Computer and System Sciences*, vol. 51, no. 2, pp. 261–272, 1995.

[2] A. Chavan, S. Rabinia, D. Grosu, and M. Brocanelli, "Speeding-up graph algorithms via clique partitioning," *Networks*, vol. 88, no. 1, pp. 85–98, July 2026.

[3] M. Bannach, F. A. Marwitz, and T. Tantau, "Faster Graph Algorithms Through DAG Compression," in *41st Int. Symp. Theoretical Aspects of Comp. Sci.*, Dagstuhl, Germany, 2024, pp. 8:1–8:18.

[4] G. Buehrer and K. Chellapilla, "A scalable pattern mining approach to web graph compression with communities," in *Proc. Int. Conf. on Web Search and Data Mining*. ACM, 2008, p. 95–106.

[5] L. Dhulipala, I. Kabiljo, B. Karrer, G. Ottaviano, S. Pupyrev, and A. Shalita, "Compressing graphs and indexes with recursive graph bisection," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, ser. KDD '16. ACM, 2016, p. 1535–1544.

[6] W. Fan, Y. Li, M. Liu, and C. Lu, "Making graphs compact by lossless contraction," in *Proc. Int. Conf. on Management of Data*, ser. SIGMOD '21. New York, NY, USA: ACM, 2021, p. 472–484.

[7] J. E. Hopcroft and R. M. Karp, "An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs," *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.

[8] D. W. Matula, "Determining edge connectivity in 0(nm)," in *Proc. 28th Annual Symp. on Foundations of Comp. Sci.*, 1987, p. 249–251.

[9] S. Even and R. E. Tarjan, "Network flow and testing graph connectivity," *SIAM J. on Comp.*, vol. 4, no. 4, pp. 507–518, 1975.

[10] M. Besta and T. Hoefler, "Survey and taxonomy of lossless graph compression and space-efficient graph representations," 2019. [Online]. Available: https://arxiv.org/abs/1806.01799

[11] M. Danisch, I. Panagiotas, and L. Tabourier, "Compressing bipartite graphs with a dual reordering scheme," 2023. [Online]. Available: https://arxiv.org/abs/2209.12062

[12] J. Shun, L. Dhulipala, and G. E. Blelloch, "Smaller and faster: Parallel processing of compressed graphs with ligra+," in *Proc. Data Compression Conference*. IEEE, 2015, pp. 403–412.

[13] Z. Chen, F. Zhang, J. Guan, J. Zhai, X. Shen, H. Zhang, W. Shu, and X. Du, "Compressgraph: Efficient parallel graph analytics with rule-based compression," *Proc. ACM on Management of Data*, vol. 1, no. 1, pp. 1–31, 2023.

[14] A. Sahebi, M. Procaccini, and R. Giorgi, "Hashgrid: An optimized architecture for accelerating graph computing on fpgas," *Future Generation Computer Systems*, vol. 162, p. 107497, 2025.

[15] C. Chen, C. X. Lin, M. Fredrikson, M. Christodorescu, X. Yan, and J. Han, "Mining graph patterns efficiently via randomized summaries," *Proc. VLDB End.*, vol. 2, no. 1, pp. 742–753, 2009.

[16] M. Adler and M. Mitzenmacher, "Towards compressing web graphs," in *Proc. Data Compres. Conf.* IEEE, 2001, pp. 203–212.

[17] Q. Wang, X. Ai, Y. Yan, S. Gong, Y. Zhang, J. Chen, and G. Yu, "Towards communication-efficient out-of-core graph processing on the gpu," *IEEE Trans. Parallel & Distrib. Syst.*, vol. 36, no. 5, pp. 961–976, 2025.

[18] H. Tan, Y. Xiao, A. C. Zhou, K. Lu, and X. Yang, "Distributed and adaptive partitioning for large graphs in geo-distributed data centers," *IEEE Trans. Parallel & Distrib. Syst.*, vol. 36, no. 6, pp. 1161–1174, 2025.

[19] S. M. Shovan, A. Khanda, and S. K. Das, "Parallel multi objective shortest path update algorithm in large dynamic networks," *IEEE Trans. Parallel & Distr. Syst.*, vol. 36, no. 5, pp. 932–944, 2025.

[20] A. Khanda, S. Srinivasan, S. Bhowmick, B. Norris, and S. K. Das, "A parallel algorithm template for updating single-source shortest paths in large-scale dynamic networks," *IEEE Trans. Parallel & Distrib. Syst.*, vol. 33, no. 4, pp. 929–940, 2022.

[21] M. Besta, M. Fischer, V. Kalavri, M. Kapralov, and T. Hoefler, "Practice of streaming processing of dynamic graphs: Concepts, models, and systems," *IEEE Trans. Parallel & Distrib. Syst.*, vol. 34, no. 6, pp. 1860–1876, 2023.

[22] A. Chavan, Par-RCP and Seq-RCP Implementation. [Online]. Available: https://github.com/aksharc2/parallelRandomCPGC.git

[23] S. T. Brown, P. Buitrago, E. Hanna, S. Sanielevici, R. Scibek, and N. A. Nystrom, "Bridges-2: A platform for rapidly-evolving and data intensive research," in *Practice and Experience in Advanced Research Computing*, ser. PEARC '21. ACM, 2021.

[24] T. A. Davis and Y. Hu, "The university of florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, Dec. 2011.

[25] C. L. Staudt, A. Sazonovs, and H. Meyerhenke, "Networkit: A tool suite for large-scale complex network analysis," *Network Science*, vol. 4, no. 4, p. 508–530, 2016.

[26] Y. Dinitz, "Algorithm for solution of a problem of maximum flow in a network with power estimation," *Doklady Akademii Nauk SSSR*, vol. 11, p. 1277–1280, 1970.

[27] J. Chuzhoy and S. Khanna, "Maximum bipartite matching in $n^{2+o(1)}$ time via a combinatorial algorithm," in *Proc. 56th Annual ACM Symp. Theory of Comp.* ACM, 2024, p. 83–94.

[28] J. van den Brand, Y. T. Lee, D. Nanongkai, R. Peng, T. Saranurak, A. Sidford, Z. Song, and D. Wang, "Bipartite matching in nearly-linear time on moderately dense graphs," in *Proc. 61st IEEE Annual Symp. Foundations of Comp. Sci.* IEEE, 2020, pp. 919–930.

BIOGRAPHIES



Akshar Chavan is a Ph.D. candidate in Electrical and Computer Engineering at The Ohio State University, USA. He received the M.S. degree in Industrial Engineering from Wayne State University, USA, in 2020, and the B.E. degree in Mechanical Engineering from the University of Mumbai, India, in 2014. His research interests include autonomous systems, energy-aware computing, mobile and edge computing, distributed and parallel systems, and scalable graph algorithms. His recent work focuses on battery health-aware autonomous systems, real-time embedded intelligence, and efficient computation for large-scale autonomous and networked systems. He is a student member of the ACM and the IEEE.



Sanaz Rabinia received her Ph.D. in Computer Science from Wayne State University in 2025. She earned her M.Sc. in Mathematics from Sharif University of Technology (Iran) in 2013 and her B.Sc. in Mathematics from Shahid Beheshti University (Iran) in 2011. Her research interests include edge computing, cloud computing, parameterized algorithms, approximation algorithms, parallel computing, and combinatorial optimization. She received an Honorable Mention for the *2024 NCWIT Collegiate Award* and the *2025 Ralph H. Kummler Distinguished Achievement Award in Graduate Student Research* from the Wayne State University College of Engineering. She is a student member of the ACM and the IEEE.



Daniel Grosu received the Diploma in engineering (automatic control and industrial informatics) from the Technical University of Iaşi, Romania, in 1994 and the MSc and PhD degrees in computer science from the University of Texas at San Antonio in 2002 and 2003, respectively. Currently, he is a professor in the Department of Computer Science, Wayne State University, Detroit. His research interests include parallel and distributed computing, graph algorithms, approximation algorithms, and topics at the border of computer science, game theory and economics. He has published more than one hundred peer-reviewed papers in the above areas and is an *IEEE Computer Society Distinguished Contributor*. He serves as a senior associate editor for *ACM Computing Surveys* and as an associate editor for *IEEE Transactions on Parallel and Distributed Systems*. He is a Distinguished Member of the ACM, a senior member of the IEEE, and the IEEE Computer Society.

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2026.3743036



Marco Brocanelli (Senior Member, IEEE) is an Assistant Professor in the Department of Electrical and Computer Engineering at The Ohio State University and the director of the Energy-aware Autonomous Systems Lab (EAS-Lab). He received the B.E. and M.E. degrees in Control Systems from the University of Rome Tor Vergata (Italy). In August 2018, he received the Ph.D. degree in the Electrical and Computer Engineering program of The Ohio State University. His research interests are in the area of cyber-physical systems, energy-aware systems, Internet of Things (IoT), edge computing, embedded and real-time systems. In 2022 he received the CAREER award from NSF and he currently serves as an associate editor for *ACM Computing Surveys*.

Authorized licensed use limited to: Wayne State University. Downloaded on October 21,2026 at 14:14:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.